Tools for Coupled Multi Physics Calculations of the Molten Salt Fast Reactor

ARFITEC Internship Report

By

Francisco Acosta

February 2016

Tutors:

Pablo Rubiolo

Mauricio Tano Retamales

LPSC Grenoble
Laboratoire de Physique Subatomique et de Cosmologie

Instituto Balseiro

ABSTRACT

In the present work, a benchmark and a coupled multi-physics calculation code based on Serpent and OpenFOAM were studied. These tools had been developed at the *Laboratoire de physique subatomique et de cosmologie* in the framework of the R&D on the Molten Salt Fast Reactor, one of the most promising Generation IV design concepts. This concept, and in particular the different phenomena involved in the coupling between neutronics and thermal-hydraulics, was first analyzed.

Then, the above mentioned code was documented and modifications needed to make it more flexible to execute the calculations involved in the benchmark were made. The benchmark was performed, and the results obtained showed that it meets its objective of providing a simulation scenario where all the fundamental coupling phenomena of the Molten Salt Fast Reactor are present and where the different elements can be progressively introduced and studied independently.

Finally, a strategy for transient calculations that conserves the Serpent + OpenFOAM approach was outlined. This strategy is based on the Predictor-Corrector Quasi-static Method with predictive time step selection and a Predict-Evaluate-Correct algorithm with higher order linearization for the lagged nonlinear coupling terms.

INDEX

1 Context and objectives of the internship

The accuracy of the calculations needed for design and safety analysis of nuclear reactors, current and future, is constantly increasing. This poses the need for more complex calculation schemes, among which the coupled multi-physics (MP) approach is most noteworthy. Within this approach, the different relevant physical components- neutronics, thermal-hydraulics and structural mechanics- are integrated to perform the required analysis, as opposed to traditional calculation schemes where these components are calculated successively and each calculation uses as input the output from its predecessor.

Great part of the efforts to develop new coupled MP calculation schemes and codes were focused on one of the most promising Generation IV design concepts: The Molten Salt Fast Reactor (MSFR). The MSFR, that will be described in more detail in the following section, uses a molten salt as nuclear fuel and has no core internal structures. The first of these characteristic makes the coupling between the different physical components tighter, and it introduces new coupling phenomena such as neutron precursors transport (Claudio Nicolino, 2008). The second one, on the other hand, implies that this reactor's geometry can be more easily modeled than the very complex configuration of solid fuel cores, and well known CFD techniques could therefore be used, even with today's computing capacity.

In the last years, in the *Laboratoire de physique subatomique et de cosmologie* (LPSC), several different tools for MP calculations were developed for the MSFR, motivated by the already mentioned challenges and possibilities it involves. Two of these tools were the starting point of this work: a coupled neutronics and thermal-hydraulic steady state calculation code based on Serpent and OpenFOAM, and a simplified benchmark designed to evaluate the performance of different codes in coupled calculations.

Associated with the aforementioned code, some needs were clearly identifiable; these constituted part of the objectives of the present internship. Firstly, it was necessary to conduct a study of the source code to be able to understand its structure and functionality, and to generate its- at the moment inexistent- documentation. Secondly, the LPSC required a running version of the code, compiled so as to run in parallel, in order to avoid excessively high calculation times. Added to this, the code had to be modified to increase its flexibility to perform the different calculations needed for the simplified benchmark, to which the second set of the internship's objective is related.

The benchmark designed in the LPSC, that will be thoroughly described in Section 3, is a simplified problem that resembles the MSFR and, therefore, it should present the most relevant physical phenomena of this reactor. To verify that this was indeed the case, it needed to be carefully studied. After this first stage, the proposed benchmark had to be performed using the coupled MP code previously explored, and the results analyzed and compared to the expected behavior from a reactor theory point of view.

The coupling approach with Serpent + OpenFOAM used in the developed code is only intended for steady state calculations, since performing any transient calculation would require an unacceptably large time. This motivated the last objective of the internship, which was to study the available bibliography on MP coupling techniques and algorithms, and to propose a possible strategy for transient coupled calculations, which would then be the point of departure of future work in this area.

2 The MSFR

2.1 Description of the MSFR design concept

The MSFR is a fast spectrum nuclear reactor that uses molten salt both as fuel and as coolant. As opposed to previous concepts with thermal neutron spectrum, the MSFR does not require a moderator such as the graphite proposed for those designs; this is why it does not have any core internal structures.

The MSFR, whose primary circuit is shown schematically in Figure 1, consists of a cylindrical vessel with a diameter and height of 2.25 m, where the fuel salt is pumped in the upward direction to later circulate downward through the heat exchangers located circumferentially around the core. It operates at ambient pressure and at a temperature of approximately 750 ºC. (Samofar, 2016)

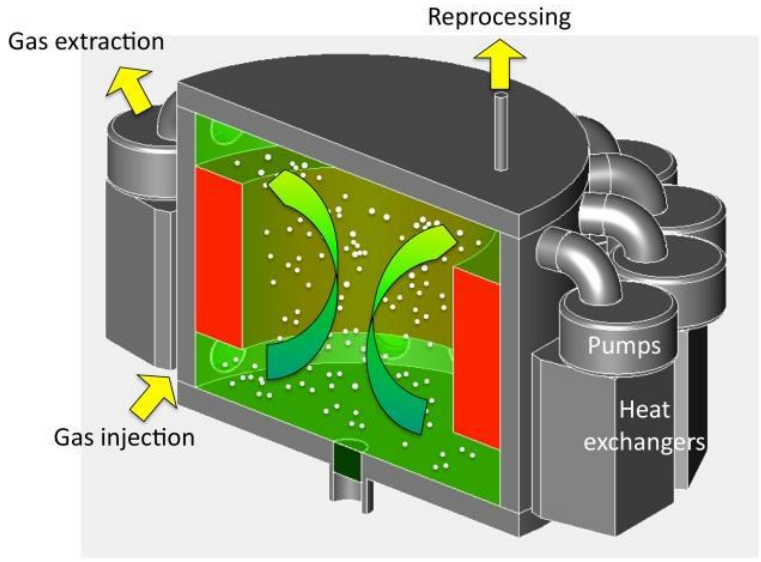

Figure 1: Schematic representation of the MSFR's primary circuit. (Samofar, 2016)

During operation, a fraction of the fuel salt is continuously diverted to a reprocessing circuit, where the fission products that are not removed by a gas bubbling process - also implemented -are extracted.

2.1.1 Advantages of the MSFR

The MSFR unique characteristics result in a series of important advantages that have made this reactor on of the six design concepts selected by the Generation IV International Forum (GIF) for further study. (OECD Nuclear Energy Agency, 2014)

Firstly, the use of liquid fuel eliminates all the mechanical complications of solid fuel, including those arising as a consequence of irradiation (PCI, swelling, etc). The reprocessing is also greatly simplified; the reactor does not even need to be stopped for it to be carried on. In addition, the homogeneous dissolution of fissile isotopes in the salt eliminates the need of a refueling plan – fairly complex in today's reactors – and the fuel's composition can be adjusted online.

Added to this, the salt's properties allow the reactor to operate at high temperatures and low pressure. This means that higher thermodynamic efficiency can be attained and smaller and lower cost confinement structures are required, since these no longer need to be able to withstand high pressure.

Furthermore, regarding the reactor's safety and in addition to the negative reactivity feedback coefficient which it shares with its solid fueled relatives, the liquid fuel provides the unique capability of easily reconfiguring the core geometry when needed. Specifically, two different core configurations are envisaged: one optimized for electricity production and a second one meant for long term storage with a passive cooling system, which could be achieved also passively by gravitational draining. Besides, control rods are no longer needed since reactivity can be controlled by the heat transfer rate in the exchangers and the thermal feedbacks. (Merle-Lucotte, 2015)

2.1.2 Current limitations of the MSFR

Even though this design concept has many promising features, the MSFR presents nowadays some limitations that motivate the efforts in R&D that are being carried on by the many different actors participating in its development.

In the first place, development and qualification of the different materials that will be used in the reactor are needed to guarantee these would be able to resist the high temperatures and irradiation to which they would be exposed. Also, further R&D is needed on liquid salt physical chemistry and technology, including corrosion, safety-related issues and treatment and reprocessing systems.

Secondly, the safety demonstration requirements will have to be completely redefined for this type of reactors, that greatly differs from any other commercially available. This last fact implies that there is no operational experience from which to profit in order to advance more rapidly with the MSFR development.

Lastly, and pointed out by the GIF in 2013 as one of the main goals in this matter of the following decade, the tighter coupling of the different physical phenomena in the MSFR require the development of advanced neutronic and thermal-hydraulic coupling models.

2.2 Physical phenomena involved and their impact in MP coupling

Not every one of the many physical phenomena that are relevant in the MSFR have an important impact on the MP coupling. This means that, even if it's necessary to include all of these in a calculation model of the reactor to obtain adequate results, some could be excluded if only the coupling is to be investigated; this is the case of the benchmark designed in the LPSC, which is the object of Section 3. A discussion of the relevance of these phenomena follows.

Fluid dynamics and turbulence

Given that the fuel itself is liquid and circulates in the core cavity, it is not a surprise that thermal-hydraulic is a critical aspect of the MSFR. Previous benchmarks and comparisons have shown that accurate Computational Fluid Dynamics (CFD) and turbulence models are needed to obtain consistent results from the calculations. (Claudio Nicolino, 2008)

The effect of fluid flow on neutronics via neutron precursor transport is well known and has a large impact. On the other hand, the direct coupling between turbulence and neutronics is expected to be fairly small due to the difference between turbulent eddy scales and neutron mean free path. This suggests that a laminar flow model might be sufficient to study the MP coupling, though this hypothesis should be carefully studied in the future.

Neutron transport

Neutronics is one of the most important physics to model correctly since it yields the power source and its tightly coupled with other phenomena, mainly through neutron precursors motion and temperature reactivity feedback.

There are several different reliable neutronics solvers available. Some are based on a stochastic approach to the neutron transport equation, like Monte Carlo codes, and produce accurate results that do not require many important approximations (Leppänen, 2009). As a drawback, these codes require large calculation times, which makes them less suitable for transient analysis if not combined with adequate speed up techniques. On the other hand, deterministic calculation codes, such as the ones based on the widely used Diffusion Approximation, are significantly faster but yield less accurate results based on sometimes strong approximations.

Geometry and 3D effects

The geometry of the reactor has a great impact both in the fuel flow and in neutronic aspects, such as the neutron leakage. Any calculation model that has as a goal to accurately represent the reactor's behavior must include a detail description of the geometry, though its symmetry could be taken into consideration to model one angular segment.

Despite this, the coupling phenomena expected in a 3D simulation are essentially the same as those expected in 2D. In other systems, where no liquid fuel mixing is possible and where there are different internal core structures and control instruments involved, spatial oscillations sustained by thermal-hydraulics – neutronics coupling might easily arise if 3D geometries are analyzed.

Precursors motion

Delayed neutron precursors motion is one of the distinctive characteristics of liquid fueled reactors. In this case, neutron precursors do not decay in the same position where they were originally produced by fission, as is the case of solid fueled reactors; thus the delayed neutrons can 'appear' in zones of lower neutron importance, or even in the heat exchangers outside the core. For this reason, precursors motion has a significant impact on the Effective Delayed Neutron Fraction, β_{eff}, which is a key parameter in the Point Kinetics model and extremely relevant for reactor safety considerations, since it determines the reactivity needed to reach 'prompt criticality'.

In fluid fueled reactors, β_{eff} differs from the physical delayed neutron fraction (β_0) for two different reason. Firstly, because the delayed neutrons are emitted with lower average energy than prompt neutrons, which is also the case in solid fueled reactors. Secondly, and as stated before, because the neutron precursors are transported by the fuel flow and may decay in lower neutron importance zones, or even outside the reactor core. This last reason is more relevant, and leads to a reduction of β_{eff}. It has been observed in the past that a correct neutron importance calculation or estimation is needed for the calculation of β_{eff} in fuel-motion conditions, to obtain accurate results. (Manuele Aufiero M. B., 2014)

Doppler effect

The Doppler effect is one of the most important temperature reactivity feedbacks mechanisms in nuclear reactors, and it is partially responsible for the core's self-stabilization following a power excursion. The resonances in the neutron absorption cross sections exhibit a broadening when temperature is increased, reducing the neutron's probability to *scape* it and thus reducing the reactivity. This effect is also coupled with thermal-hydraulics due to the fact that fuel temperature depends on the power density distribution and on the flow conditions.

However, the Doppler effect is not a distinctive characteristic of the MSFR, in which, besides, the temperature feedback through density changes is of the same magnitude (Merle-Lucotte, 2015). Including in the calculation model only the feedback through fuel density changes is enough to represent the tightly coupled MP scenario of the MSFR, and it eliminates the need of including several ad-hoc neutronic capabilities for the evaluation of the Doppler effect, such as resonance broadening, self-shielding evaluation, non-resolved resonance treatment, etc.

Density effect

The density effect is one of the most relevant temperature reactivity feedback in the MSFR. The fuel density is strongly dependent on neutronics, which provides the heat source distribution, and on thermal-hydraulics, responsible for the fuel flow and velocity profile.

The main effect of fuel density variation is to modify neutron leakage. As temperature rises, density decreases and some fuel salt is 'pushed' outside the core's cavity. This reduces de neutron macroscopic cross sections, that affect fission and absorption rates equally. Nevertheless, it also increases the neutron leakage, increasing the transport of neutron to lower importance zones, thus reducing reactivity.

Thermo-mechanical effects

Salt solidification, propagation of pressure waves and other thermo-mechanical phenomena are especially relevant in fast transients, and should be taken into consideration for safety analysis. Nonetheless, they do not represent a strong two-way coupling with neutronics in most of the normally evaluated scenarios or transients.

2.3 MP coupling in the MSFR

Most of the phenomena described so far are also present in solid fueled reactors. However, in the MSFR the coupling between them is tighter, and effects such as precursors drift are only observed in the latter case.

The MP coupling present in the reactor is schematically described in Figure 2, where the heat generation in the fuel salt is also showed. It can be noted that fission affects the temperature distribution directly through prompt energy. Added to this, fission products are transported by the fuel salt motion and, as they decay, the decay heat also affects temperature. The temperature distribution, in turn, affects the neutron absorption and leakage rate through the Doppler effect and density changes. This impacts the neutron flux distribution, which is also influenced by the delayed neutrons that result from the decay of neutron precursors. Finally, the fission rate is determined by the neutron flux, and the coupling cycle starts all over again.

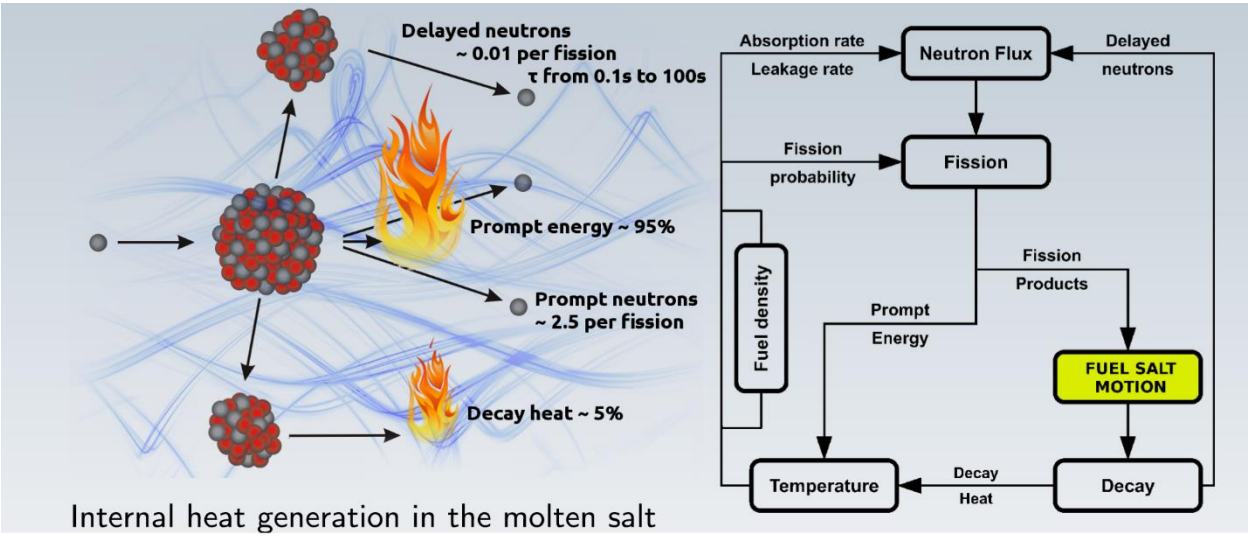

Figure 2: Schema of the internal heat generation in the molten salt and of the MP coupling in the MSFR. (Manuele Aufiero A. C., 2012)

3 A simplified benchmark for MP calculation codes

One of the tools developed in the LPSC for the MP coupled calculations for the MSFR is a simplified benchmark, which is described in this section.

3.1 Objectives of the benchmark

The main objective of the benchmark is to compare the performance of the physical models used for the neutronic – thermal-hydraulic coupling and their numerical implementation, focusing in the MSFR study case. It must be noted that it is not meant to be another benchmark for stand alone, single-physics CFD or neutronic calculation codes, since there are several others designed with that exact purpose. In addition, the proposed benchmark does by no means try to replace code validation and verification efforts, that should be conducted in parallel.

For the previous reasons, the benchmark aims to asses and compare the capabilities of different codes of representing the complexity of the MP coupling scenario of the MSFR rather than to provide a calculation exercise that accurately represents the reactor.

3.2 Benchmark's main features

Before specifying any details of the benchmark's design, a few key characteristics that it should have were defined in the LPSC.

Fluid fuel

As mentioned before, reactors with liquid fuel present a tighter MP coupling, and new effects connecting the different components arise. Added to this, the reduced geometrical complexity, when compared with solid fuels, makes it easier to focus on the MP coupling models. For these reasons, the benchmark should incorporate fluid nuclear fuel.

Similarity with the MSFR

It was sought that the main physical phenomena in the benchmark had similar relative weight than what they have in the MSFR. For this to be the case, certain parameters such as the dimensions of the simulated problem, the composition of the fuel, the neutron spectrum, etc, should be adjusted to achieve an acceptable similarity with the MSFR.

Inclusion of most relevant physics of the MSFR

All of the relevant physical phenomena present in the reactor that have a significant impact in the MP coupling, already discussed in Section 2.2, should be also present in the benchmark. Therefore, oversimplification of the problem should be avoided.

Simple geometry

Given that the benchmark is meant to be a tool used to asses and compere the performance of different MP coupling models and codes, it is desirable that the largest amount of codes available can perform it. Hence, the geometry to be simulated should be kept as simple as possible, so that no calculation code is left out due to its lack of capability to represent the proposed configuration.

Progressiveness

One of the fundamental features that the benchmark was meant to have is the capability of providing different calculation scenarios so that the origin of the potential discrepancies in the results obtained with different codes could be easily identified. For this reason, the benchmark should present several different calculation stages, throughout which the complexity and the number of phenomena included in the simulation should be progressively increased.

3.3 Description of the simplified problem

Taking into account all the considerations so far developed, the system to be modeled in the benchmark was specified. It consists of a molten salt fuel, whose composition is indicated in Table 1,circulating in a 2D, 2 m sided square cavity, that is shown in Figure 3. The cavity has an upper moving wall with a fixed velocity Ux, and no slip condition is adopted in all walls. This, together with the buoyancy forces also taken into account, determine the velocity profile of the salt. Void boundary condition is imposed for the neutron flux in all walls; this means that no neutron reflector was modeled. In addition, the walls were considered adiabatic and null pressure gradient was forced in them.

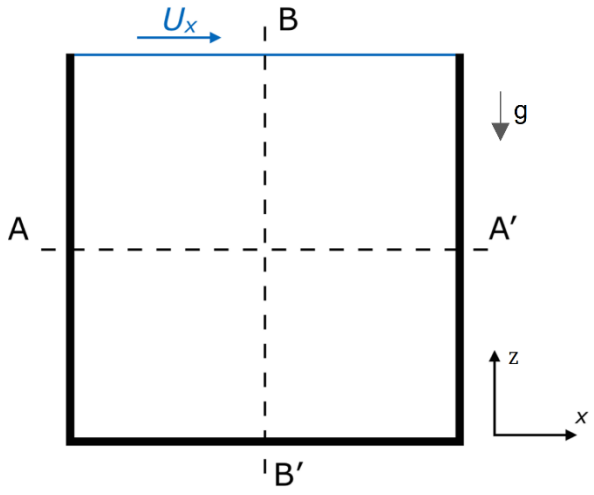

Figure 3: Scheme of the modeled cavity. (Laureau, 2015)

In order to balance the power generation resulting from nuclear fission and thus reach a stationary state, heat extraction is modeled through a constant heat exchange coefficient $h = 10^6\ \frac{\mathrm{w}}{\mathrm{m^2K}}$ (or $\frac{\mathrm{w}}{\mathrm{m^3K}}$ if 1 m depth in the *y* direction is assumed) and with a medium at constant temperature $T_{ref} = 900\ \mathrm{K}$. Therefore, the removed power density takes the form $Q_{removal}(x,z) = h(T(x,z) - T_{ref})$.

The fluid flow is considered to be laminar, since turbulence-neutronic coupling is assumed to have lesser significance. It is also considered incompressible, and the Boussinesq approximation is used to include buoyancy forces due to density changes. All other thermophysical properties are kept constant, and are specified, together with other relevant parameters, in Table 2.

Delayed neutron precursors drift is modeled, and 8 different families, whose decay constants and fractions are shown in Table 3, are used. The precursors groups parameters are prescribed in the benchmark in order to reduce variability between calculations performed with different codes, that would result from using different family constants. For this reason, the nuclear data library is also prescribed (JEFF-3.2 in this case), as well as the few-energy-groups constants that would be needed for neutron diffusion calculations that might be carried by some codes.

Table 1: Composition of the fuel salt modeled.

Isotope	Li 6	Li 7	F 19	Be 9	U 235
Mass fraction	15 (2.11%)	185 (26.08%)	400 (56.40%)	100 (14.10%)	9.259 (1.31%)

Table 2: Relevant parameters of the modeled problem.

Parameter	**Notation and units**	**Value**
Laminar viscosity	$\mu\ [\mathrm{m^2/s}]$	0.025
Thermal expansion coefficient	$\beta_{expansion}\ [1/\mathrm{K}]$	2e-4
Reference temperature	$T_{ref}\ [\mathrm{K}]$	900
Laminar Prandtl number	Pr	3.075e5
Turbulent Prandtl number	Prt	0.85
Laminar Schmidt number	Sc	2e8
Turbulent Schmidt number	Sct	1
Density times heat capacity	$\rho C_p\ [\mathrm{kg/ms^2K}]$	6.15e6
Gravity acceleration	$g\ [\mathrm{m/cm^2s}]$	9.81
Heat exchange coefficient	$\mathrm{h}\ [\mathrm{w/m^2K}]$	1e6
Reference thermal power level	$P_{ref}\ [\mathrm{GW}]$	3

Table 3: Fractions and decay constant of the 8 neutron precursor families modeled.

Family	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
$\boldsymbol{\beta_f}$ **[pcm]**	24.3	103.0	68.1	137.4	214.4	64.0	59.9	6.5
$\boldsymbol{T_{1/2}}$ **[s]** $(\frac{\boldsymbol{ln(2)}}{\boldsymbol{\lambda}})$	55.6	24.5	16.3	5.21	2.37	1.04	0.424	0.195

3.4 Benchmark's different stages

The benchmark is divided in three different phases. A first one to verify the single physics capabilities of the code being analyzed, a second one with several different steps that progressively include more phenomena in the simulation, and a third one for transient calculations. The first two phases have been studied and performed in the present work, and are described in the following sections. (LPSC, 2014)

3.4.1 Phase 0: Preliminary single physics verification

In phase 0, the single physics calculation capabilities of the codes are verified. It is, of course, mandatory to obtain consistent results in this preliminary stage before proceeding with the rest of the benchmark. In addition, the results obtained in this phase will be used to normalize some of the results of Phase 1, and will also, in some cases, be used as fixed inputs.

Step 0.1: Velocity field

In this step, the solution of the steady state, incompressible flow in the 2D geometry is studied. No heat source is considered, and uniform temperature T_{ref} is adopted. The only momentum source considered is the upper moving wall, whose velocity is fixed at 1 m/s.

As previously discussed, a correct solution of the fluid flow is needed to obtain consistent MP coupling results. Therefore, any discrepancy obtained using different codes at this early stage should be addressed before proceeding with the following steps, some of which, in addition, will use as external fixed input the velocity field of this step.

The main observables, this is, the physical variables that will be used as a reference result for quantitative and qualitative comparisons, are the velocity components along the line AA' (see Figure 3), the velocity in the 2D geometry, and the flow streamlines.

Step 0.2: Neutronics

In this step, the eigenvalue neutronic solution is analyzed for the case of static fuel. The neutron flux level is normalized to a total power P_{ref}, but no link between neutronics and thermal-hydraulics is considered; this means that the temperature was set to T_{ref} in the whole domain.

The objective of this step is to verify the neutronic solution of the different codes in the simplest configuration possible. The same philosophy than in the previous step applies when it comes to discrepancies encountered in this early point between different codes: these must be addressed before continuing with the benchmark.

To obtain a representative value of the effective delayed neutron fraction β_{eff}, both energy and spatial effects must be taken into account. In this step, the energy effects can be evaluated by verifying the correct neutron importance weighting of the delayed neutrons, for the codes that have such capabilities. A reference value $\beta_{eff,static}$ is stablished.

The main observables of this step are the fission rate density along AA' and in the 2D geometry, the effective multiplication factor and the effective delayed neutron fraction.

Step 0.3: Temperature

In this step, the temperature distribution calculation is verified. The velocity field from step 0.1 is used and kept constant, as well as the power density distribution from step 0.2. Therefore, the scalar transport capabilities of the code employed are assessed separately and independently from the neutronics and fluid dynamics. Any problem detected in this stage of the calculation would also impact the neutron precursors transport that is included in some of the steps of Phase 1. As described in 3.3, the heat removal needed to reach a steady state is modeled through a constant heat exchange coefficient.

The main observables of this step are the temperature distribution along AA', BB', and in the 2D geometry.

3.4.2 Phase 1: Steady-State coupling

In Phase 1 the neutronic – thermal-hydraulics coupling is analyzed. Different phenomena, such as buoyancy, precursor transport or thermal feedbacks on neutronics, are progressively introduced in the different steps of this phase, in order to facilitate the identification of the origin of the discrepancies obtained with different codes.

Step 1.1: Circulating fuel

The velocity field from Step 0.1 is imposed, as well as uniform temperature field $T(x,z) = T_{ref}$. In these conditions, the neutron flux profile and the neutron precursors concentrations are obtained, for a fixed power P_{ref}.

The objective of this step is to assess the correct evaluation of the effects of the fluid flow on neutronics, particularly the reactivity loss due to fuel motion. This is expected because the fuel's motion transports the neutron precursors to lower neutron importance zones, where they decay; this is not the case with static fuel, where delayed neutrons are produced where their precursors where produced by fission.

The main observables of this step are the delayed neutron source ($\sum_i \lambda_i C_i$) in lines AA' and BB', as well as in the 2D geometry. Added to this, the reactivity change from step 0.2 is observed, together with the effective delayed neutron fraction normalized with the value obtained in that step.

Step 1.2: Parametric study with circulating fuel

The moving wall's velocity Ux is varied to investigate the effect of fuel motion on reactivity, for a wide range of flow conditions. Uniform T_{ref} is kept.

The reactivity change and the normalized delayed neutron fraction as functions of the velocity of the top wall are observed.

Step 1.3: Power coupling

Once again, the velocity field of Step 0.1 is adopted. In this case, a uniform volumetric heat removal coefficient is used to model the heat extraction needed to reach a steady state, for which the temperature distribution is obtained together with the neutron flux profile and neutron precursors concentrations.

The *two-way* coupling between neutronics and thermal-hydraulics is investigated for the simple case of fixed velocity field. The effect of the temperature distribution on the neutron flux shape can be evaluated.

The observables of this step are the temperature distribution along AA', BB' and on the 2D geometry, the reactivity change from step 1.1, and the change of fission rate density with respect to the step 0.2.

Step 1.4: Parametric study with power coupling

Under the conditions of Step 1.3, power level is varied to investigate its effect in the coupling. For higher power levels, the flux deformation due to the temperature field is greater. This may lead to inaccurate results for some neutron calculation codes to be evaluated and, if this is the case, this step should make this evident.

The reactivity change from step 1.1 as a function of the power level is observed.

Step 1.5: Buoyancy

In this step, the full neutronics/thermal-hydraulic coupling is analyzed, with null velocity for the upper wall. Therefore, only buoyancy forces generated by the temperature gradients are responsible for the fluid's motion.

The capability of the codes to predict the correct velocity field induced by the fission source is evaluated. Since most multi-physics coupling phenomena have been studied in the previous steps, discrepancy in the results obtained with different codes can be considered mainly related to the buoyancy effects.

In this step, the velocity, temperature and delayed neutron source distributions along AA', BB' and in the 2D geometry are observed. Added to this, the flow streamlines obtained with uniform seeds sampling in the AA' are evaluated. The reactivity change from step 0.2 and the effective delayed neutron fraction are also studied.

Step 1.6: Parametric studies with buoyancy effects

Under the same conditions than in the previous step, the total power P is varied. The buoyancy effects increase with increasing power, thus the coupling becomes stronger and may induce some inaccuracies in more approximate calculation codes; this step is aimed to the identification of these potential inaccuracies.

The main observables of this steps are the reactivity change from step 0.2 and the normalized effective delayed neutron fraction, as functions of the power level.

Step 1.7: Parametric studies with full coupling

This step involves the solution of the complete multi-physics problem, and the velocity and temperature fields, as well as the neutron flux and precursors concentrations distributions, are calculated. All previously analyzed phenomena are here included: external momentum source, buoyancy effect, delayed precursor motion and temperature effects on the neutron flux shape.

The calculation is repeated for different power levels, keeping Ux=U_{ref}, and for different velocities Ux of the moving wall, maintaining P=P_{ref} in this case.

This step is thought as representative of a realistic simulation of the MSFR, following the general guidelines stablished in Section 3.2. This step will ultimately be used for the final comparison of the different calculation codes, if they had produced consistent results in the previous steps.

The observables of this step are the reactivity change from step 0.2 and the normalized effective delayed neutron fraction, this time as functions of both power level and velocity of the top wall.

3.4.3 Phase 2: Transient analysis

This phase of the benchmark seeks to assess the coupled MP calculation capabilities of different codes in transient scenarios. This is of great importance in safety analysis, where postulated accidental situations, such as fast reactivity introductions, are simulated.

Phase 2 of the benchmark is not explored in the present work, mainly due to the limitations of the code employed, which is described in the following section. One of the objectives of this internship was,

precisely, to outline a strategy that could be employed to overcome those limitations, as described in Section 1. This outline is developed in Section 6.

4 Code employed to perform the benchmark

One of the main goals of the internship was to perform the benchmark, using for this purpose a neutronics + thermal-hydraulics code based on the coupling of Serpent and openFOAM, developed in the LPSC. In this section, both Serpent and OpenFOAM are first briefly introduced. Then, the coupled problem to be solved is described, to later show how the coupling is implemented. Finally, a user-oriented description of the input/output system of the code is given.

The coupled code was compiled to run in parallel, using the OpenMP platform, in a Linux environment.

4.1 Serpent Monte Carlo code

The behavior of neutrons in the reactor is ruled by the Boltzman neutron transport equation, which can be written for the angular flux $\psi(\boldsymbol{r}, \boldsymbol{\Omega}, E, t)$ in position $\boldsymbol{r}$, oriented in direction $\boldsymbol{\Omega}$, at energy E and at an instant t as

$$\begin{aligned}\frac{1}{v}\frac{\partial\psi(\boldsymbol{r},\boldsymbol{\Omega},E,t)}{\partial t} = &-\boldsymbol{\Omega}\cdot\nabla\left(\psi(\boldsymbol{r},\boldsymbol{\Omega},E,t)\right)-\Sigma_t(\boldsymbol{r},E)\psi(\boldsymbol{r},\boldsymbol{\Omega},E,t)\\ &+\int_0^\infty \mathrm{d}E'\int_{4\pi}\mathrm{d}\boldsymbol{\Omega}'\Sigma_s(E'\to E,\boldsymbol{\Omega}'\to\boldsymbol{\Omega})\psi(\boldsymbol{r},\boldsymbol{\Omega}',E',t)\\ &+\frac{1}{4\pi}\int_0^\infty \mathrm{d}E'\nu_p(\boldsymbol{r})\Sigma_f(\boldsymbol{r},E')\chi_p(E'\to E)\phi(\boldsymbol{r},E',t)\\ &+\sum_f\frac{1}{4\pi}\lambda_f P_f(\boldsymbol{r},t)\chi_{d_f}(E)\end{aligned}$$

This equation represents a balance between neutron loses, with the transport term and the total interactions, and the neutron gains, with the inscattering, the promt fission source and the delayed neutron source.

In reactor calculations, there exist two distinct approaches to the neutron transport equation. One is deterministic, and it is based on the discretization of time, space and energy and, in some cases, on strong approximations; such is the case of the widely used Diffusion method. The other approach is stochastic, and the Boltzman equation is not explicitly solved in this case, but a Monte Carlo technique is used to obtain an approximate solution.

The Monte Carlo technique basically consists on simulating many different neutron “histories”, this is, from the neutron production until its absorption or leakage from the system. In these simulations, that can be run independently due to the null neutron-neutron interaction, probability distributions are used to determine the interaction of the modeled neutron with the surrounding medium. These distributions are representative of the physical problem, and no strong approximations are made to model the neutron behavior. Therefore, the results obtained are statistical estimators that become more accurate with a growing number of histories run and, if this number is adequate, are generally more accurate than the results obtained with deterministic methods.

Serpent is a three-dimensional continuous-energy Monte Carlo reactor physics burnup calculation code, developed at VTT Technical Research Centre of Finland (Leppänen, 2007). It features two

different simulation modes: k-eigenvalue criticality source and external source mode; only the first one is relevant for the present work.

In the k-eigenvalue criticality source method, the simulation is run in cycles and the source distribution of each cycle is formed by the fission reaction distribution of the previous one. The number of source neutrons per cycle is fixed and, since the source neutrons generated during the simulation is not, the source size is either decreased or increased to match the specified number in the cases of supercritical and subcritical systems, respectively.

When the criticality source method is started, a source term is randomly distributed throughout the modeled geometry, and a guess for the effective multiplication factor is introduced by the user. Since the initial guess for the source is not physically representative, a number of "inactive cycles" is run so that the fission source converges, before starting to collect statistics for the results.

The statistical accuracy of the results depends on the total number of active neutron histories run, which is determined by the number of neutrons per cycles and the number of cycles. It can be shown that the error in the estimators, measured using the standard deviation, is inversely proportional to the square root of the number of histories run: $\sigma \propto \frac{1}{\sqrt{N}}$. This evidences how costly it is to reduce the uncertainty of the results; for example, to reduce it by 10% the number of histories simulated should be multiplied by 100.

The probability distributions of the neutron path length, that are needed for simulating the neutron history, must be computed within a homogeneous region. In nuclear reactor calculations, the geometry to me modeled is divided in small cells, each of them with homogeneous properties. Usually, Monte Carlo codes have to calculate the distance of the neutron being simulated to the next cell surface, all along its track length. If the mean free path is large compared to the cells sizes, this calculations - as well as the re-calculation of the probability distributions within each cell - must be done several times for each neutron history, which may result in high computational cost. In the Serpent Monte Carlo code, however, this tracking algorithm is replaced by an alternative method, known as delta-tracking, when it is convenient to do so.

The delta-tracking algorithm is based on adding to each material an appropriate cross section of "virtual collisions", that do not affect the neutron that undergoes them in any aspect, so that every cell in the modeled geometry has the same total cross section. By doing this, the need of recalculating the free path length within each cell is eliminated, as well as the necessity of computing the distance to the cell surfaces.

In coupled neutronic/thermal-hydraulics calculations, the space discretization used for the thermal problem determines a great number of cells with different properties, such as density or temperature, that can be interpreted as different materials regarding their interaction with the neutrons. The efficiency of the delta-tracking algorithm makes it possible to use for the neutronic calculations the same space discretization than for the thermal-hydraulic problem, without penalizing the performance of the code; this leads to a simplification in the information exchange strategies between the different physical components. (JaakkoLeppanen, 2014)

4.2 OpenFOAM

OpenFOAM is, above all, a C++ library, used to create executables known as *applications.* Applications can either be classified as *solvers*, which are designed to solve a specific problem in continuum mechanics, or *utilities*, that perform task involving data manipulation. Numerous solvers and utilities are included in the OpenFOAM distribution, including several for CFD applications, particularly relevant for this work. (OpenFOAM: The Open Source CFD Toolbox. User Guide, 2015)

One of the great advantages of OpenFOAM is that users can create new solvers that are specifically tailored for a given problem, making use of all the tools and capabilities of the library for solving partial differential equations, as well as the friendly syntax developed for this purpose. Added to this, custom objects, such as boundary conditions or turbulence models, can be created without modifying the existing source code. It also has parallel computation capabilities based on the domain decomposition technique. (OpenFOAM: The Open Source CFD Toolbox. Programmer's Guide, 2015)

OpenFOAM is based on the finite volume method, and arbitrarily shaped cells can be used; i.e. with any number of faces and any number of edges. When equations governing a given problem are coupled, and particularly for the pressure and velocity equations in CFD calculations, adapted versions of well-known algorithms such as PISO and SIMPLE are used.

Finally, the distribution includes a post-processing tool called ParaFOAM, which enables the user to visualize the geometry being modeled as well as the fields calculated. It is also capable of performing data manipulation.

4.3 Definition of the coupled problem

The problem specified in Section 3.3 is solved using the OpenFOAM environment for the thermal-hydraulic component and the neutron precursor transport, and Serpent Monte Carlo for the neutronic component.

Incompressible flow is modeled, therefore the incompressible continuity equation takes the form

$$\nabla \cdot \vec{u} = 0 \tag{1}$$

To take into account buoyant forces, the Boussinesq approximation is used, which leads to the following momentum equation

$$\frac{D\vec{u}}{Dt} = -\frac{1}{\rho_0}\nabla p + \frac{\mu}{\rho_0}\nabla^2\vec{u} - \frac{\rho\vec{g}}{\rho_0}, \tag{2}$$

where

$$\rho = \rho_0\left[1 - \alpha\left(T - T_{ref}\right)\right]. \tag{3}$$

The energy equation can be written as

$$\frac{DT}{Dt} = k\nabla^2 T + \frac{Q_{fission} - Q_{removal}}{\rho_0 C_p}. \tag{4}$$

The fission heat source is obtained from the fission rate estimator in Serpent, R_f, and it is calculated as

$$Q_{fission} = E_f \nu \Phi \Sigma_f = E_f \nu R_f, \tag{5}$$

where E_f is the energy released per fission and ν the average number of neutrons released per fission.

The heat extraction, on the other hand, is modeled through a constant volumetric heat exchange coefficient h and is calculated in every point as

$$Q_{removal} = h\left(T - T_{ref}\right). \tag{6}$$

Finally, the equation that determines the neutron precursors concentration C_i can be expressed, for the precursor family *i*, as

$$\frac{DC_i}{Dt} = D\nabla^2 C_i - \lambda_i C_i + Q_i, \quad (7)$$

where

$$Q_i = \beta_i \nu \Phi \Sigma_f = \beta_i \nu R_f. \quad (8)$$

The previous set of equations manifest the coupled nature of the problem. Firstly, it can be noted that the fission heat source is proportional to the neutron flux, and that it determines the temperature distribution; this can be seen in the energy equation. Within the Boussinesq approximation, temperature affects density in the gravity term of the momentum equation, thus affecting the velocity distribution, which, in turn, affects the convective derivatives in the energy and precursors equations. This has an impact on the neutron flux firstly trough the thermal feedback coefficients, which are not explicitly shown in the equations, and, secondly, through the delayed neutrons.

4.4 Coupling strategy

The coupling strategy adopted is based on an iterative scheme where information about each cell in the geometry is exchanged between the neutronic calculation code, Serpent, and the CFD code, implemented in OpenFOAM. In particular, the main transport routine in Serpent makes several calls to solvers implemented in OpenFOAM for the thermal-hydraulic equations, and for the neutron precursors equations. To perform a steady state calculation, OpenFOAM solvers are run multiple times, each of which uses as input an updated fission heat source calculated with several consecutive Serpent runs; the process is repeated until convergence, normally determined by the change in the velocity profile from one iteration to the next one.

The original Serpent transport routine, which was later slightly modified in this internship, is summarized in the pseudocode presented in the Figure 4. This routine is called **oftransportcycle.c**, and, for the sake of simplicity, only some of the most relevant procedures it comprises are included here.

The transport routine is run once for every cycle or batch of neutrons. After reaching the user introduced number of initial cycles used to converge the neutron source distribution, *#skip*, the stored statistics for the results are cleared, as shown in line 10 of Figure 4. In every transport cycle, the source is first normalized to keep the neutron population constant. After this, the tracking procedure (**oftracking()**) is run for every neutron in the cycle. It is within this function that Serpent gets the information about the different cells in the geometry generated with OpenFOAM; specifically, the function **OFSampleMaterial()** takes into consideration the temperature and **OFDensityFactor()** the density. After **oftracking()** is run for every neutron in the cycle, the effective multiplication factor is calculated, as seen in line 22 of the same figure.

The next steps of the routine involve the resolution of the thermal-hydraulic and neutron precursors equations. This is done every 50 cycles by calling the function **ofsolvephysics**, in line 26. In this function, the volumetric heat source is updated using the reaction rates estimators from the transport cycle in Serpent. Finally, an iterative procedure is conducted to solve for the velocity (**UEqn.H**), the pressure (**pEqn.H**), the temperature (**TEqn.H**) and the neutron precursors concentrations (**SolvePrecursors.H**). The results produced in this stage are used by Serpent when the function **oftracking()** is run for the next cycle.

This algorithm has a number of limitations regarding its flexibility when it comes to performing, for example, the calculations needed for the benchmark. As detailed in Section 3.4, there are several different steps that require that one or more of the physical components are not updated with every

step of the simulation which, in practice, means not running one of the specific solvers. Added to this, some steps require that the heat source is not updated. For this reason, the code was modified so that 3 Boolean variables are read from one of the input files, specified in Section 4.5.3, and determine whether **UEqn** + **pEqn** and **TEqn** should be included or not (i.e. if the momentum equation and energy equation solvers should be called or not), and if the volumetric heat source *volpower* is updated or kept constant. This can be observed in lines 32, 38 and 28, respectively, of the modified pseudocode presented in Figure 5, where it is noted that these Boolean variables are *flag_update_u*, *flag_update_t* and *flag_update_source.*

Another limitation arises from the fact that the number of transport cycles between two consecutive OpenFOAM runs was fixed to 50, and this could only be changed by modifying and recompiling the source code. Such a large number of consecutive transport calculations is not always necessary to guarantee the fission source convergence, and if a lower number is used the code's runtime could be greatly reduced. Furthermore, even in the case that no transport calculations are needed, such as that of Step 0.3 of the benchmark, 50 transport cycles had to be run for every OpenFOAM run, with the great time requirement this implies. To tackle these issues, the code was modified so that it reads from an input file two new scalar variables: *numbatch* and *numof.* The modified code runs the OpenFOAM solvers *numof* consecutive times every *numbatch* transport cycles, as shown in the lines 24 and 21 of Figure 5, respectively.

```
 7 Loop over all batches
 8     {
 9
10         if(i=#skip) delete statistics; %% Source converging before this point
11
12            NormalizeCritSrc(); %% Adjust #neutrons in generation to keep k=1
13
14          Loop over source neutrons
15            {
16            oftracking(); %% Main neutron tracking routine.
17                          %% Includes OFSampleMaterial() that imports cell data from
18            }             %% OpenFoam, to take T into account and OFDensityFactor()
19                          %% to consider density
20
21
22            IterateKeff();
23
24            if(i%50==0) %% Every 50 cycles
25                {
26                    ofsolvephysics     %% SIMPLE iterative algorithm implemented
27                                       %% to solve for U and P. T and precursors
28                                       %% are also solved.
29                    {
30                       Update volpower %% Update the volumetric heat source
31                       Loop over UTPcorrector
32                        {
33                          #include "UEqn.H" %% Velocity equation
34                          #include "pEqn.H" %% Pressure equation
35                          Loop over Tcorrectors
36                            {
37                                #include "TEqn.H" %% Energy equation
38                            }
39                        }
40                       Loop over PRECcorrectors
41                        {
42                          #include "solvePrecursors.H" %% Precursors equations
43                        }
44                    }
45                }
46     }
```

Figure 4: Pseudocode of Serpent's main transport routine, before it was modified. Some of the most relevant functions are included.

```
Loop over all batches
        {

            if( i=#skip) delete statistics;

            NormalizeCritSrc();

            Loop over source neutrons
            {
            oftracking()
            }

            IterateKeff();

            if(i%numbatch==0) %% Every numbatch cycles
                {

                Repeat numof times %% OpenFOAM is run numof consecutive times
                          {
                               ofsolvephysics
                                   {
                                      if(flag_update_source==1) update volpower;

                                      Loop over UTPcorrector
                                           {
                                               if(flag_update_u!=0)
                                                    {
                                                      #include "UEqn.H"
                                                      #include "pEqn.H"
                                                     }

                                               if(flag_update_t!=0)
                                                   {
                                                     loop over Tcorrectors
                                                        {
                                                          #include "TEqn.H"
                                                        }
                                                    }
                                           }
                                   }
                          }
                        Loop over PRECcorrectors
                             {
                               #include "solvePrecursors.H"
                             }
                    }
          }
```

Figure 5: Pseudocode of Serpent's main transport routine, after it was modified. Some of the most relevant functions are included.

4.5 General Input/Output description

The code is run within a Case Directory which, in turn, contains the following sub directories:

- **Serpent Directory:** Serpent input/output files.
- **System Directory:** OpenFOAM input/output files.
- **Constant Directory:** Constant properties of the modeled system.
- **Initial State Directory:** Initial value of all neutronic and thermal-hydraulic variables involved.

A description of these directories and the most important files they contain follows. This description is by no means comprehensive, and special attention is paid to the aspects that are more relevant for running the benchmark cases and for interpreting the results.

4.5.1 Serpent Directory

All Serpent's input and output files are stored in this directory. For a case named *NameCase*, the more relevant files found in this directory are *NameCase* and *NameCase_res.m.*

The *NameCase* file contains all the information that Serpent needs for the calculations, such as the composition of the materials involved, the description of the geometry, the path where to look for the cross section library, the boundary conditions for the neutronic problem, the thermal power level and, of extreme importance, the input card *set pop.*

The *set pop* card, whose syntax is *set pop <npop> <cycles> <skip> [<keff0> <int>]*, determines the number *npop* of source neutrons per cycle, the number *cycles* of active cycles run, the number *skip* of inactive cycles run, the initial guess *keff0* for the effective multiplication factor and the number *int* of generations run for each batch of results. This card is essential for controlling the results' accuracy and the total runtime of the code. It is important to note that *cycles* is the number of cycles of the complete calculation, and not the number run between two OpenFOAM runs. This means that if the user considers that, for example, 500 cycles are needed to achieve the desired accuracy in the fission source and knows that OpenFOAM will be run 100 times in total, *cycles* should be set equal to 500*100+50=50050, where 50 *skip* cycles were also accounted for.

An example of the file *NameCase* used for one of the steps of the benchmark is shown in Figure 6. It can be noted that the geometry is just defined as a cube, in line 3, and no information about the cell discretization is given. The reason for this is that Serpent uses the discretization imposed by OpenFOAM, which, besides simplifying this input file, makes data exchange between the codes easier. Serpent's user manual (Leppänen, 2015) should be consulted for further details of the *NameCase* file.

```
 1 set title "bench1u5"
 2
 3 surf s1 cuboid -100.0 100.0 -50.0 50.0 -100.0 100.0
 4
 5 cell c1 0 fuel -s1
 6 cell out 0 outside s1
 7
 8 mat fuel        -2.02020202020202
 9 %Li             200
10 Li-6.09c        15
11 Li-7.09c       185
12 F-19.09c       400
13 Be-9.09c       100
14 U-235.09c        9.259
15
16 % --- Cross section library file path:
17 set acelib "/7gpr/aufiero/xsdata/jeff311/sss_jeff311u.xsdata"
18
19 set bc 1 2 1
20 % set pop <npop> <cycles> <skip> [<keff0> <int>]
21
22 set pop 10000 141000 100
23
24 set seed 123456789
25
26 set opti 3 %server
27
28 set power 3000000000 % 3 GWth
29
30 set ures 0
31
32 set gcu -1
```

Figure 6: Example of a NameCase file used in the benchmark's calculations.

On the other hand, the file *NameCase_res.m* contains the results outputted by Serpent by default. The results that the user could have requested by adding a card for this purpose in the *NameCase* file are outputted in different files.

Of all the parameters present in the *NameCase_res.m* file, the more relevant ones for the benchmark's calculations are the effective multiplication factor k_{eff} and the effective delayed neutron fraction β_{eff} and, for both of them, different estimators are given.

The implicit estimator of k_{eff} was used in the present work to analyze the results. This choice, however, is not of extreme importance since the estimators given do not normally differ in more than 1 pcm. On the contrary, differences of a few percent were found between the β_{eff} estimators. Previous studies have shown that for fluid fueled systems the best results are obtained with the Iterated Fission Probability (IFP) method (Manuele Aufiero M. B., 2014), thus the estimator calculated with this method (DJ_IFP_ANA_BETA_EFF) was used in this work.

4.5.2 System Directory

The System Directory contains three different files that are used as input by the routines implemented in OpenFOAM. In the *fvSolution* file, the solvers for the different thermal-hydraulic fields are specified, and some of their parameters, such as convergence tolerance or maximum number of iterations, are established. The *fvSchemes* files sets the numerical schemes for terms, such as derivatives in equations, that appear in the applications being run. The last file in the System Directory is *controlDict*, which contains the input data needed for the time control and the reading and writing of the solution data.

All the steps of the benchmark involved steady state calculations. Nevertheless, the successive iterations needed to converge to the steady state, starting from the initial conditions entered by the user, can be interpreted as the time evolution of the system that is reaching that steady state; though the successive steps would not accurately represent the real time dependent behavior due to the relaxation factors used in the solvers. This analogy evidences the fact that, even in steady state calculations, the time step δt that is defined in *controlDict* is a relevant parameter. In fact, it must be chosen carefully to achieve numerical stability, and this is done by keeping the Courant number (C_o) below unity. This requirement, that must be fulfilled for each cell of the simulated geometry, is expressed as

$$C_o = \frac{\delta t \left| U \right|}{\delta x} < 1, \tag{9}$$

where $\left| U \right|$ is the modulus of the velocity and δx is the cell size in its direction. This implies that the time step has to be reduced when either the velocity is increased or the cell size is reduced.

An example of a *controlDict* file used in one of the steps of the benchmark is shown in Figure 7, where it can be seen that the time step is 0.005 s; this time step was used for all the calculations performed in this work. Other relevant parameters, such as the starting simulation step or the number of steps between solutions are written, are defined in this file.

Detailed information about the files in the System Directory can be found in OpenFOAM User's Guide (CFD Direct Ltd., 2015).

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM Extend Project: Open Source CFD        |
|  \\    /   O peration     | Version:  1.6-ext                               |
|   \\  /    A nd           | Web:      www.extend-project.de                 |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      controlDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

application     moltenSaltAdjointBoussinesqSimpleFoamModel;

startFrom       latestTime;
//startFrom       startTime;

startTime       0;

stopAt          endTime;
//stopAt          writeNow;

endTime         20;

deltaT          0.005       ;

writeControl    timeStep;

writeInterval   20

purgeWrite      0;

writeFormat     ascii;

writePrecision  9;

writeCompression uncompressed;

timeFormat      general;

timePrecision   6;

runTimeModifiable yes;

// ************************************************************************* //
```

Figure 7: Example of a controlDict file used for one of the benchmark's calculations.

4.5.3 Constant Directory

The Constant Directory contains a series of files that specify constant properties of the system being modeled. These files are:

- *transportProperties:* specifies transport properties such as viscosity, density, etc.
- *RASProperties:* specifies the turbulence model and the flow regime.
- *pumpHexProperties:* specifies the reference temperature T_{ref} and the volumetric heat exchange coefficient *h*.
- *g:* specifies the gravity acceleration.
- *decayConstants:* specifies the decay constants of the 8 different families of neutron precursors modeled.

- *globalParameters:* specifies the total thermal power and, after the code was modified, it includes the three Boolean variables that control which variables are updated in the simulation and the two control parameters *numbatch* and *numof*, discussed in Section 4.4.

An example of *globalParameters* is shown in Figure 8, where it can be noted that, in this particular case, only the temperature distribution is calculated (*flag_update_t=1*) and that the OpenFOAM solvers are run 1 time every 50 batches (*numof=1* and *numbatch=50*).

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.3.0                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.org                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    location    "constant";
    object      globalParameters;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

p_tgt           p_tgt [ 1 2 -3 0 0 0 0 ] 3e+09;

p_tot           p_tot [ 1 2 -3 0 0 0 0 ] 3e+09;

flag_update_source flag_update_source [ 0 0 0 0 0 0 0 ] 0;

flag_update_u   flag_update_u [ 0 0 0 0 0 0 0 ] 0;

flag_update_t   flag_update_t [ 0 0 0 0 0 0 0 ] 1;

numbatch        numbatch [ 0 0 0 0 0 0 0 ] 50;

numof           numof [ 0 0 0 0 0 0 0 ] 1;
```

Figure 8: Example of a globalParameters file, used in one of the benchmark's steps.

In addition, the Constant Directory contains the subdirectory polyMesh, which, in turn, contains files that describe the geometry being modeled as well as the definition boundary conditions for the thermal-hydraulic simulation. A detailed description of these files can be found in OpenFOAM User Guide. For the simple geometry of the benchmark, a square grid of 200*200 elements was used, and a uniform expansion ratio of 2.5 was implemented; this means that a cell in the boundary of the square is 2.5 times smaller than one in its center, and that cell sizes vary uniformly along the *x* and *z* directions.

4.5.4 Initial State Directory

The Initial State Directory, that takes the name "0", contains the initial values that the different variables involved in the simulation take for each cell of the geometry. The boundary conditions for these variables are also selected here, using the classes defined in the polyMesh directory. The initial conditions for any given variable is defined in an independent file.

When performing calculations is often useful to use, as starting point, a distribution for the involved fields that is closer to the expected results than what, for example, a uniform field would be. In order to do this, the corresponding files in the Initial State Directory could be replaced with the files from

some previous simulation that is similar to the one being performed; this leads to a reduced convergence time.

Some steps of the benchmark require that certain variables, such as temperature or velocity, are not updated. In this case, the user should correctly specify the Boolean variables in the *globalParameters,* file located in the Constant Directory, so that the fields defined in the Initial State Directory are not modified during the simulation. It is important to note that special care should be taken when the velocity is the field that is not to be updated. When this is the case, not only the appropriate initial condition for the velocity should be specified, but also for the derived field *Phi,* that OpenFOAM calculates and represents the velocity in the cells' boundaries.

5 Benchmark's results

The most relevant results obtained for each of the steps of the Benchmark are presented and discussed in this section.

5.1 Phase 0 results

5.1.1 Step 0.1

The streamlines associated to the velocity field are shown in Figure 9, where the velocity magnitude is also shown. The maximum velocity is 1 m/s and corresponds, as expected, to that of the moving upper wall, due to the fact that no slip condition was imposed in the cavity walls. The recirculation of the salt within the cavity is correctly modeled. The axis orientation shown is used to present the results of all the other steps of the benchmark.

The velocity profile obtained in this step was used as fixed input for several of the steps that follow.

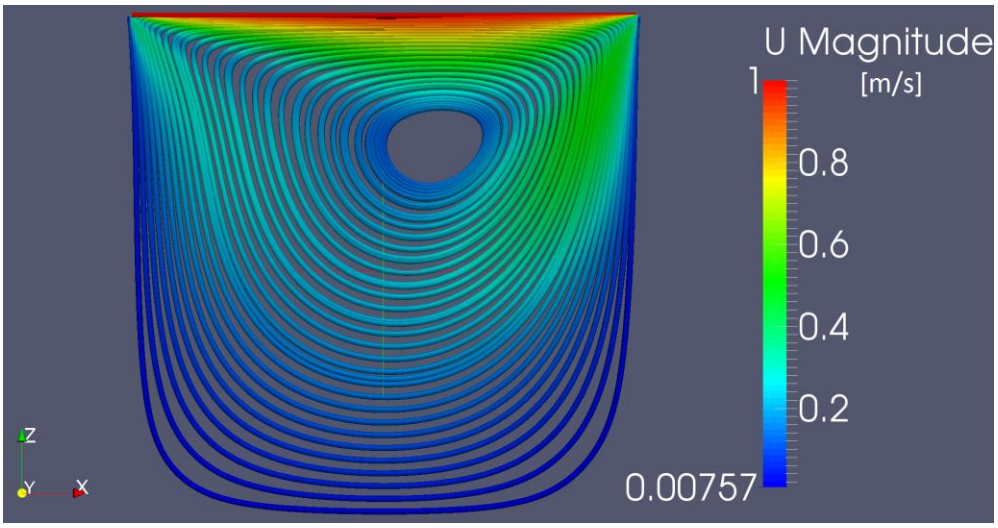

Figure 9: Streamlines of Step 0.1. The magnitude of the velocity is shown.

5.1.2 Step 0.2

The fission rate density obtained for this step is shown in Figure 10. The distribution observed is consistent with what is predicted for the neutron flux by, for example, the Neutron Diffusion Theory: a double cosine distribution that dies of in the cavity's boundaries.

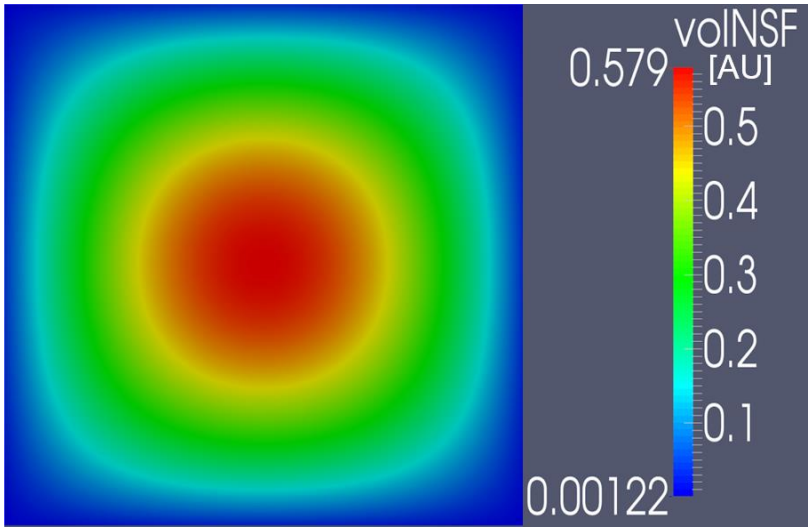

Figure 10: Fission rate density for the static, isothermal case. Arbitrary units are used.

The effective multiplication factor obtained for this configuration, that is used as a reference value, is $k_{eff}^{static} = 1.0022$, which results in a reactivity of $\rho_{static} = 219.5$ pcm. The effective delayed neutron fraction obtained with the IFP method is $\beta_{eff}^{static} = 654.562$ pcm, which is reported here as found in Serpent's output file so that it can be compared with other results in the future.

The system is less than 220 pcm away from criticality under this conditions because the salt composition was adjusted to this end, so that a reactivity reference is established in this step.

5.1.3 Step 0.3

The temperature distribution obtained for this step is shown in Figure 11. The distribution observed is a consequence of the velocity profile of step 0.1, shown in Figure 9, and the heat source of step 0.2, whose distribution is proportional to that shown in Figure 10. In this case, even if the highest heat density is in the central area of the cavity, the zone with higher temperatures is shifted to the region where the velocity is lower; i.e., towards the upper left corner of the cavity.

It can be noted that the maximum temperature exceeds 2150 k, which is much higher than the 750 ºC at which the MSFR is supposed to operate. For the purpose of investigating how suitable the benchmark is to evaluate and compare coupled calculations capabilities of different codes, simulating such high temperatures is not a problem; on the contrary, it intensifies the coupling phenomena. Nevertheless, for evaluating calculation and design tools specifically designed for the MSFR, it might be desirable that the benchmark provides a scenario that more closely resembles that of the reactor. If this is the case, the temperature range can be matched with that of the MSFR by, for example, reducing the power level or increasing the heat exchange coefficient; both alternatives would lower the maximum temperature.

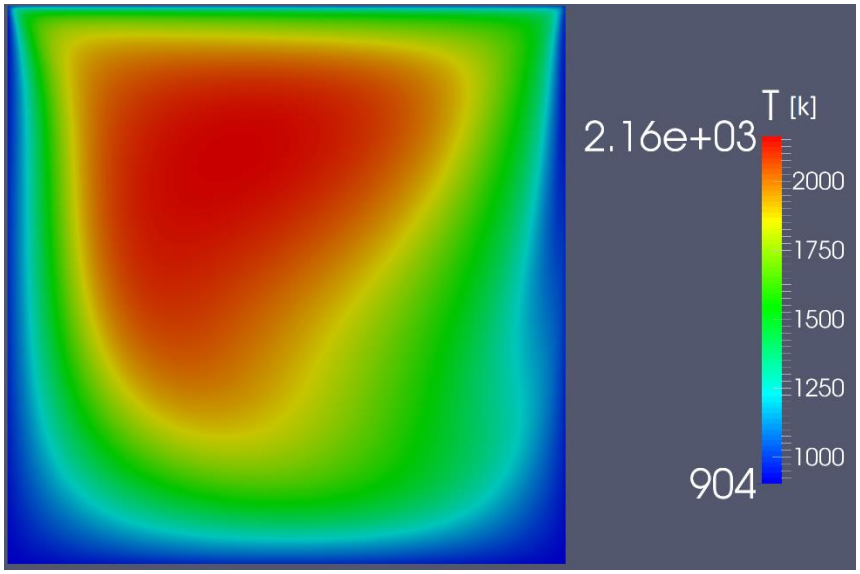

Figure 11: Temperature distribution when velocity field from step 0.1 and heat source from step 0.2 are used as input.

5.2 Phase 1 results

5.2.1 Step 1.1

In this step, the effects of fluid flow on neutronics are introduced for the first time. The reactivity calculated is $\rho = 147.7$ pcm, which results in a decrement from the static reference of $\rho_{static} - \rho = 71.7$ pcm. As explained in Section 3.4.2, one of the goals of this step is to verify the reactivity reduction due to the transport of neutron precursors to lower neutron important regions, where they decay. For the same reason, the precursors drift also affects the effective delayed neutron fraction, which in this case resulted in $\beta_{eff} = 627.025$ pcm; this leads to $\beta_{eff}/\beta_{eff}^{static} = 0.96$.

Without fuel movement, all neutron precursor families have the same distribution: that of the neutron flux in the cavity, proportional to the one presented in Figure 10. Nevertheless, fuel flow does not affect al precursor groups in the same way. The ones that decay faster will do so in a position close to that where they were produced by fission, while the slowing decaying ones might be transported for a considerable time before they give place to a delayed neutron. This situation can be observed in Figure 12 and Figure 13, where the distribution of the fastest and slowest decaying precursor families are, respectively, shown. It is there clear that the distribution of the slowest decaying precursor group presents the greatest departure from the static case.

The neutron precursors drift yields the delayed neutron source distribution shown in Figure 14, where it can be noted that the departure from the static case is somewhere between that shown in Figure 12 and the one presented in Figure 13. This depends, of course, on the relative fractions and decay constants of the precursor groups, presented in Table 3.

An important remark needs to be made about all results concerning the neutron precursors distribution. This is that, even if the obtained distributions are qualitatively correct, and the results evidence the different behavior of the different precursor families, these are not to be taken as a numerical reference for future comparisons. The reason for this is an error encountered in the code: the eight equations for the precursors concentrations share the same source term, instead of having different ones as defined in Equation 8.

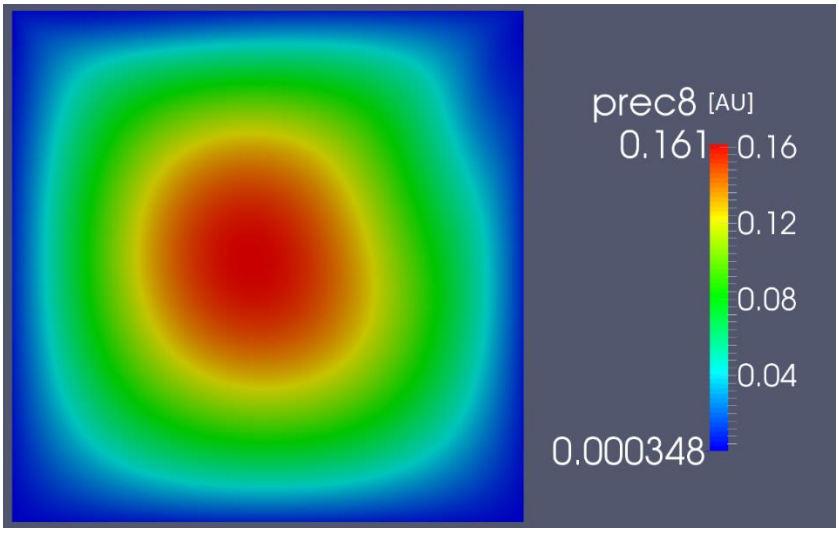

Figure 12: Distribution of the fastest decaying neutron precursor family, with $\boldsymbol{t_{1/2} = 0.195\ s}$.

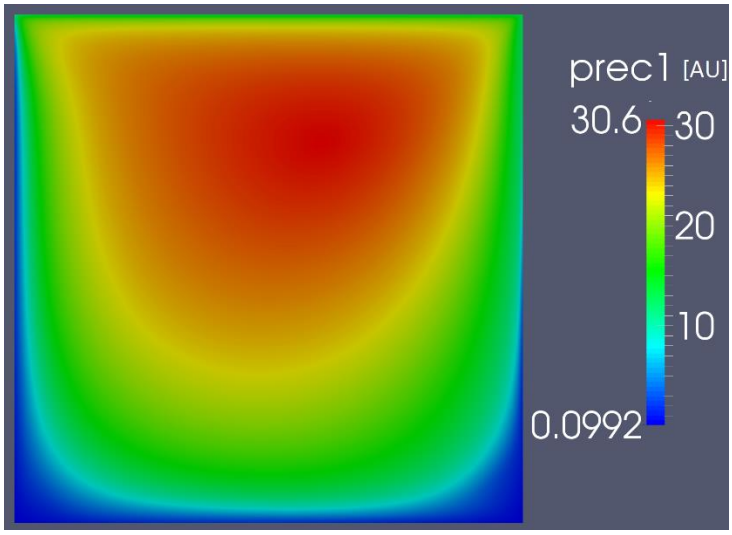

Figure 13: Distribution of the slowest decaying neutron precursor family, with $\boldsymbol{t_{1/2} = 55.6\ s}$.

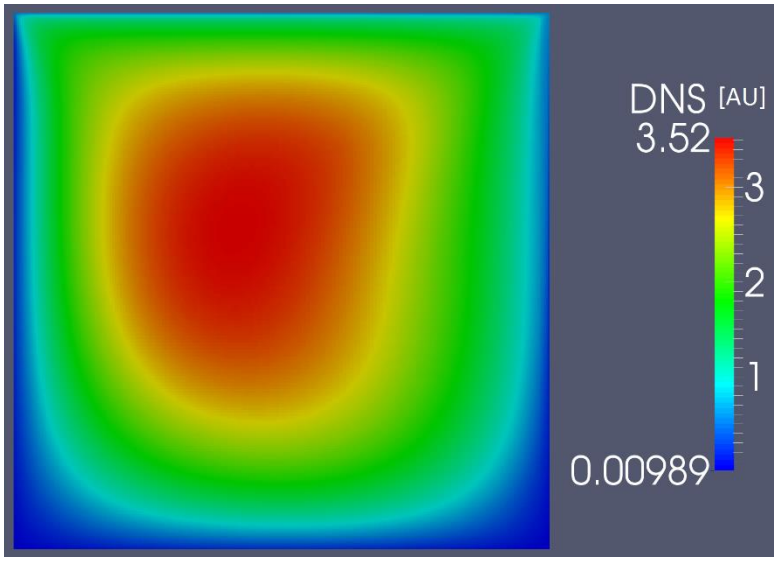

Figure 14: Delayed neutron source distribution. Uniform temperature and flow driven by moving wall only.

5.2.2 Step 1.2

When the velocity of the moving wall is changed while fixing a uniform temperature profile in the cavity, the only factor that has an impact and modifies neutronics is the precursor drift. Thus, in this scenario both the reactivity and the effective delayed neutron fraction monotonically decrease for increasing moving wall speed, for the reasons already developed in previous sections. This is shown in Figure 15 for ρ and in Figure 16 for β_{eff}.

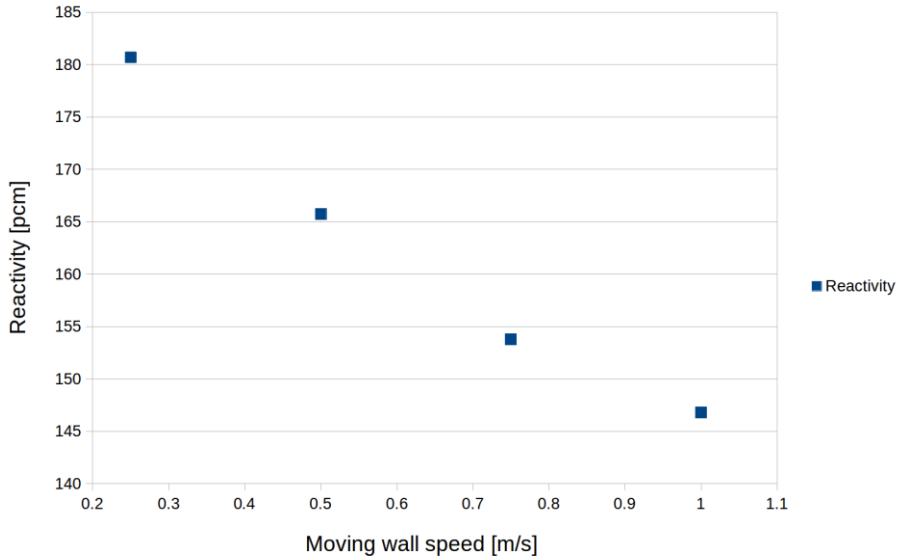

Figure 15: Reactivity as a function of the moving wall speed, for uniform temperature distribution.

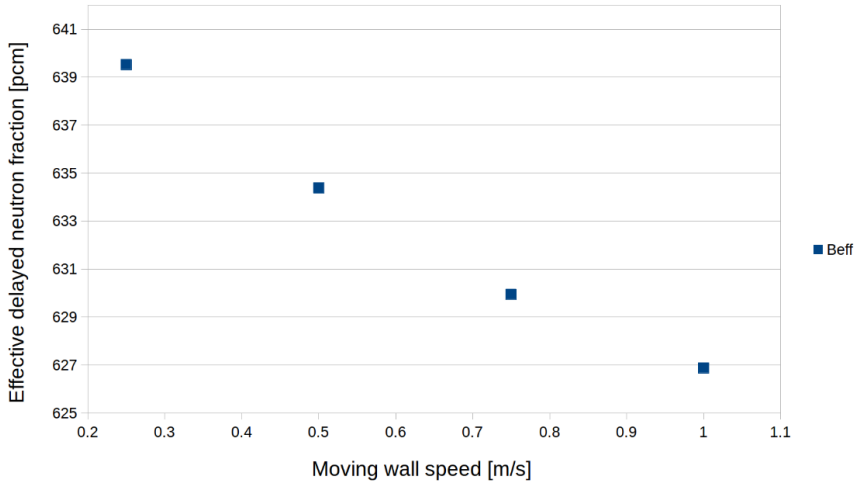

Figure 16: Effective delayed neutron fraction as a function of the moving wall speed, for uniform temperature distribution.

5.2.3 Step 1.3

The non-uniform temperature profile induces a deformation in the flux shape. The temperature distribution calculated for this step, whose reactivity resulted in $\rho = -3925.15$ pcm, is shown in Figure 17.

In the regions with higher temperature, the lower density increases neutron leakage and, thus, decreases neutron flux. This effect can be seen in Figure 18, that presents the relative percentage difference in the fission rate density between the case of uniform temperature and the case with the temperature distribution shown in Figure 17. Since the power level is kept constant, the total fission rate must also be conserved. Therefore, there central region of higher temperatures, where the fission rate density is decreased, is compensated with the periphery, where an increment of almost 40% is observed.

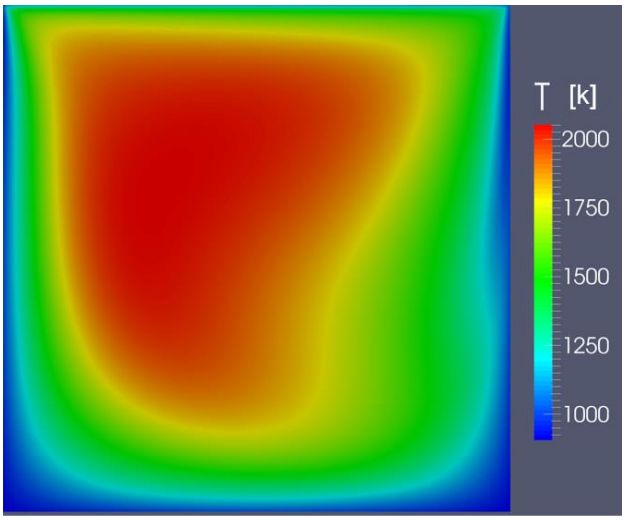

Figure 17: Temperature distribution obtained for the case with power coupling.

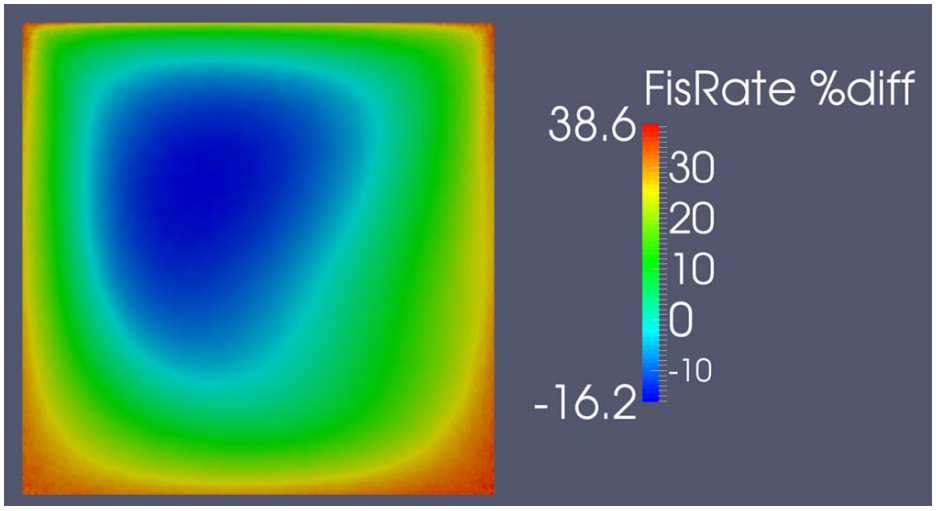

Figure 18: Relative percentage change in the fission rate density when a non-uniform temperature profile is considered.

5.2.4 Step 1.4

In this case the power level is increased but the velocity profile of Step 0.1 is fixed. Under these conditions, the only effect that has an impact and changes reactivity is the temperature feedback, which is strongly negative. This results in the monotonic decrement of reactivity with increasing power level, as presented in Figure 19.

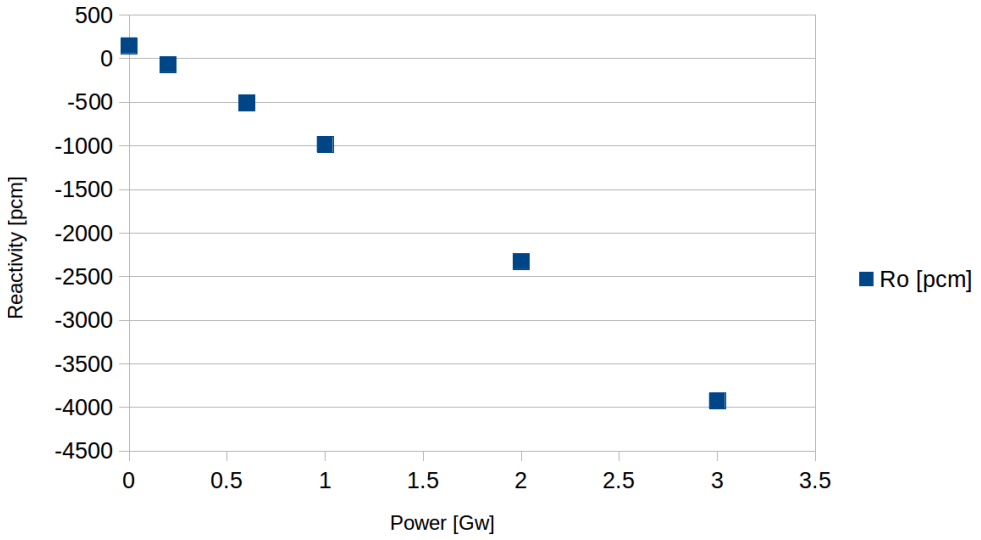

Figure 19: Reactivity as a function of power level, for the power coupling case.

5.2.5 Step 1.5

In this step, the buoyancy effects are introduced for the first time, and they are the only driving force of the fluid flow. The velocity profile obtained for this case is shown in Figure 20, where it can be noted that the maximum velocity of the fluid, for nominal power level, is about one third of that achieved with the nominal moving speed wall. This maximum is no longer in the upper part and with horizontal direction, but is reached on both sides of the cavity where the flow recirculates in a downward direction.

The fuel at higher temperature, thus with lower density, is transported upwards by the buoyant forces, leading to the temperature distribution presented in Figure 21.

The velocity field presented in Figure 20, with two distinct recirculation loops, transports the neutron precursors leading to the delayed neutron source distribution shown in Figure 22. This distribution also presents two *lobes* of higher density, that arise from a combination of high fission rate (high precursors production) and low fluid velocity (low precursor transport).

For this step, the reactivity and the effective delayed neutron fraction calculated are $\rho = -3937.84$ pcm and $\beta_{eff} = 632.144$ pcm. This leads to $\beta_{eff}/\beta_{eff}^{static} = 0.966$, where the reduction is not only a consequence of the transport of precursors to lower neutron importance areas but to the upward shift of the temperature profile as well.

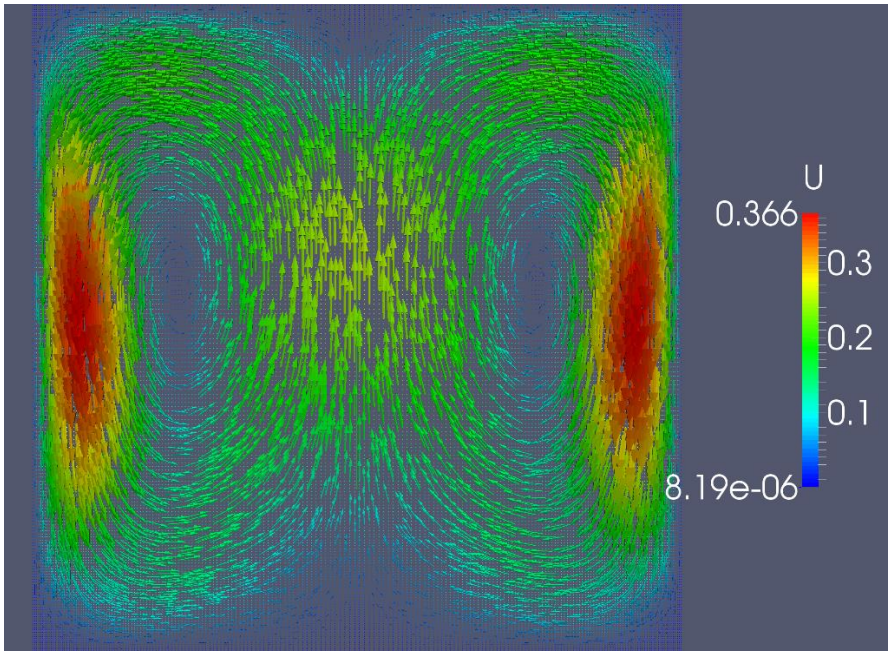

Figure 20: Velocity profile when fluid flow is driven only by buoyant forces, for the nominal power level $\boldsymbol{P_{ref}}$.

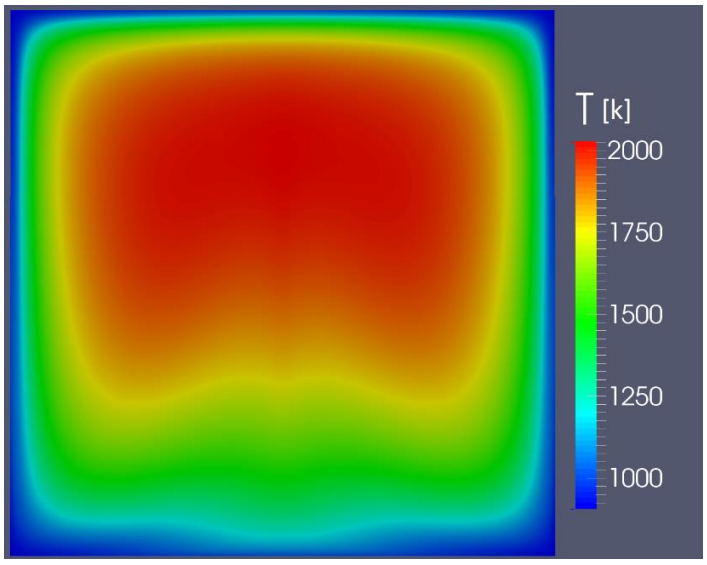

Figure 21: Temperature distribution for the case of buoyancy driven flow.

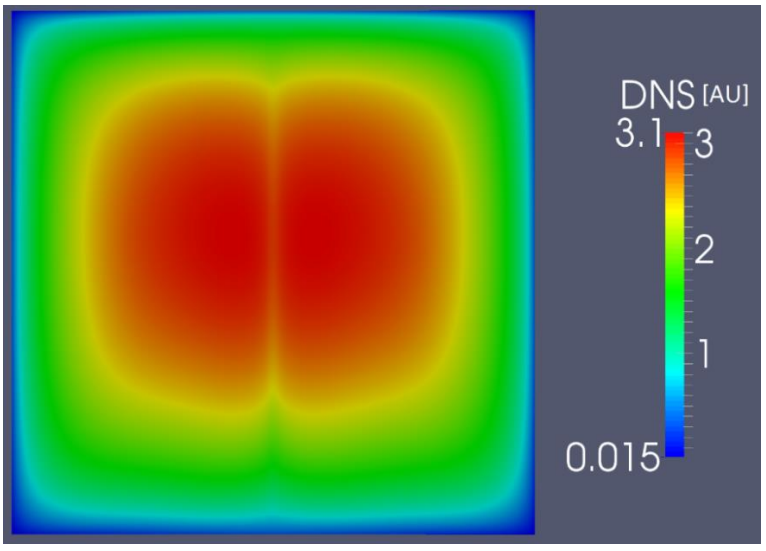

Figure 22: Delayed neutron source distribution for the case of buoyancy driven flow.

5.2.6 Step 1.6

As power is increased, and thus temperature and fuel flow rate increase as well, reactivity decreases. The reactivity as a function of the power level is presented in Figure 23. In this case, and as opposed to that of Step 1.4 where the velocity field was fixed, the fuel flow increases with power level, since the buoyant forces become stronger. Nevertheless, the evolution of reactivity as power is increased are almost identical in both cases (see Figure 19), since differences smaller than 30 pcm were found. This evidences the dominant role of the temperature in the MP coupling.

The effective delayed neutron fraction also decreases with increasing power, though this trend is deaccelerated as the nominal power is reached. The results obtained are presented in Figure 24, where this effect can be observed. For low fuel circulation velocities, the precursors that are generated in the regions with high fission rate density, and high neutron importance, are transported and may decay in low importance zones. These velocities are not large enough to allow the precursors to recirculate and decay in the same region where they had been originally produced, leading to significant reduction in β_{eff}. As fuel flow is increased, some of the longest lived precursors are recirculated and decay in the same region where they had been produced by fission, or in a region of similar neutron importance. At this point, the reduction in β_{eff} as a consequence of an increment in power level is smaller than when no precursor is transported long enough to recirculate. This is the reason behind the deceleration of the trend observed in Figure 24, especially in the step from 2 to 3 GW.

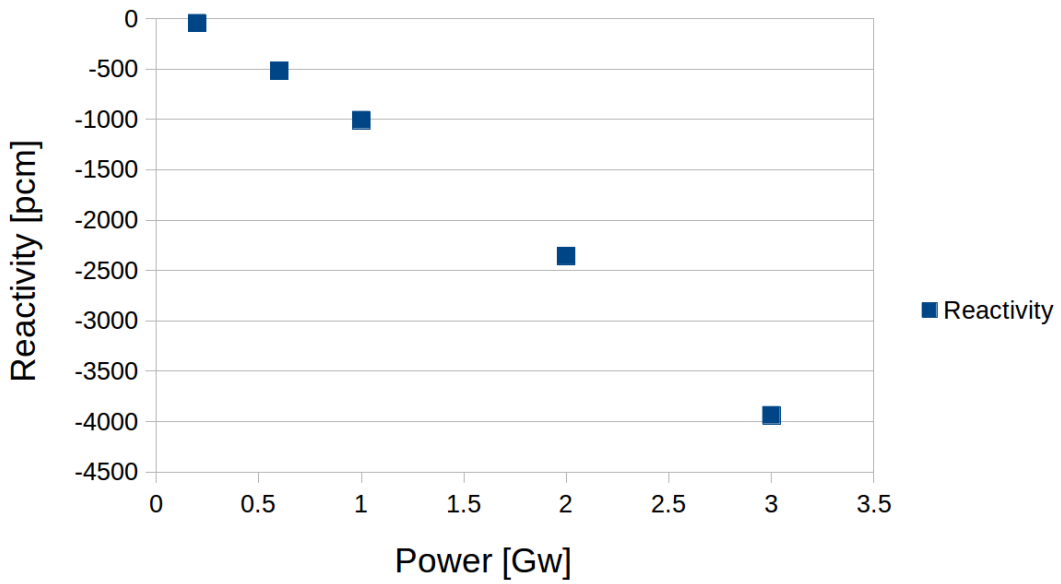

Figure 23: Reactivity as a function of the power level, for the case of buoyancy driven flow.

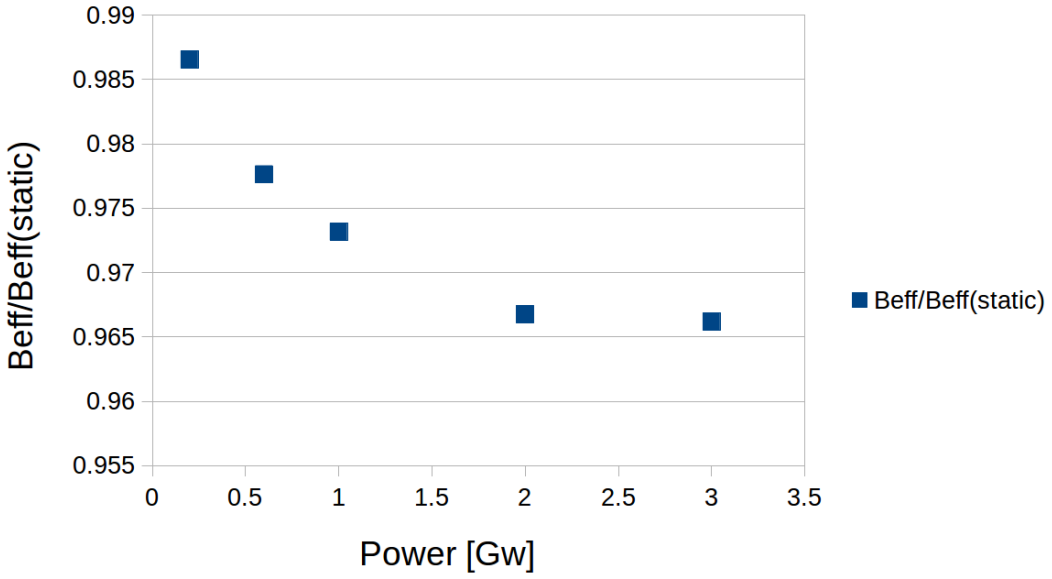

Figure 24: Normalized effective delayed neutron fraction as a function of the power level, for the case of buoyancy driven flow.

5.2.7 Step 1.7

When all components of the coupled MP problem are simultaneously introduced, effects that are not present when those components are separately evaluated arise. This is the case of the relationship between reactivity and the moving wall speed, shown in Figure 25. In this case, reactivity increases as the wall moves faster, while the opposite behavior was observed in Step 1.2 (see Figure 15). The difference lies in the temperature distribution, that was uniform in Step 1.2 but is not in this one. As the moving wall speed increases, the temperature profile is shifted in the direction of the wall's movement, as can be observed in Figure 26. This, in turn, leads to higher reactivity than that that would be obtained with the distribution of the static case (i.e. one that closely resembles the neutron flux shape). The loss in reactivity that results from precursor drift, that becomes larger for higher speeds, is overpowered by the reactivity gain due to temperature's distribution shift.

On the other hand, the precursor drift is of vital importance for the β_{eff}, whose evolution as the moving wall speed is increased is presented in Figure 27. It is noted that it decreases with increasing wall speed, and no deceleration in this trend is observed for the calculated velocities.

The reactivity and effective delayed neutron fraction are presented, as functions of the power level, in Figure 28 and Figure 29, respectively, and their behavior is qualitatively the same as in the buoyant driven flow case (Figure 23 and Figure 24). The reactivity and β_{eff} decrease with increasing power, and this trend shows a deceleration in the case of β_{eff} . However, a slightly higher reactivity is observed in this case, due to the temperature profile shift that results from higher fluid circulation. In addition, lower values were obtained for β_{eff} as a consequence of increased precursor transport that arises when the external momentum source is included in the simulation.

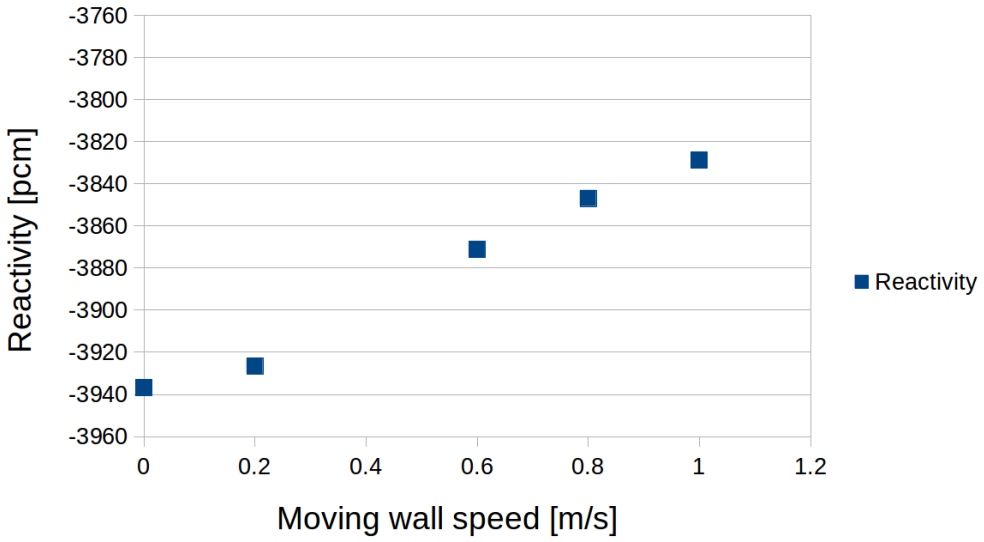

Figure 25: Reactivity as a function of the moving wall speed, for the fully coupled case.

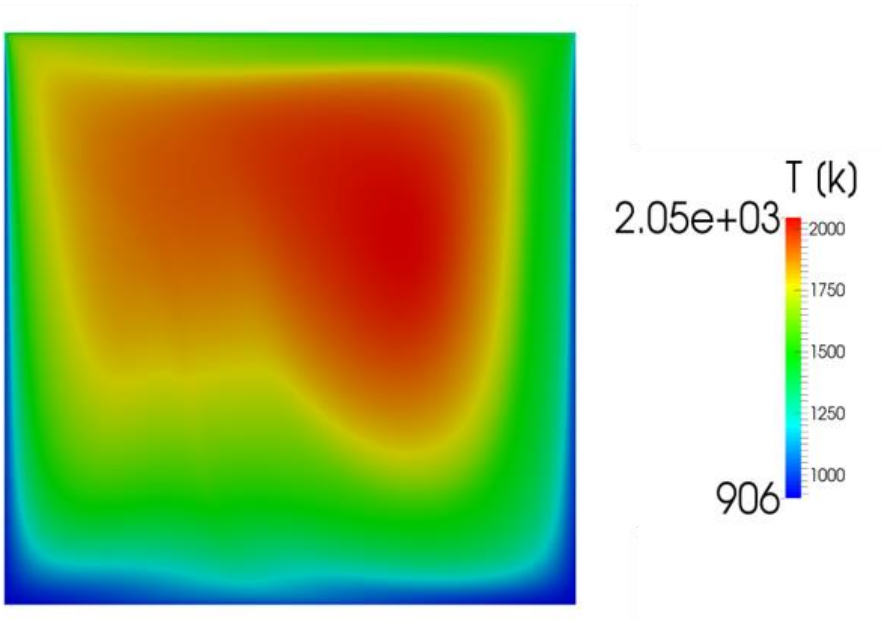

Figure 26: Temperature distribution for the full coupled case.

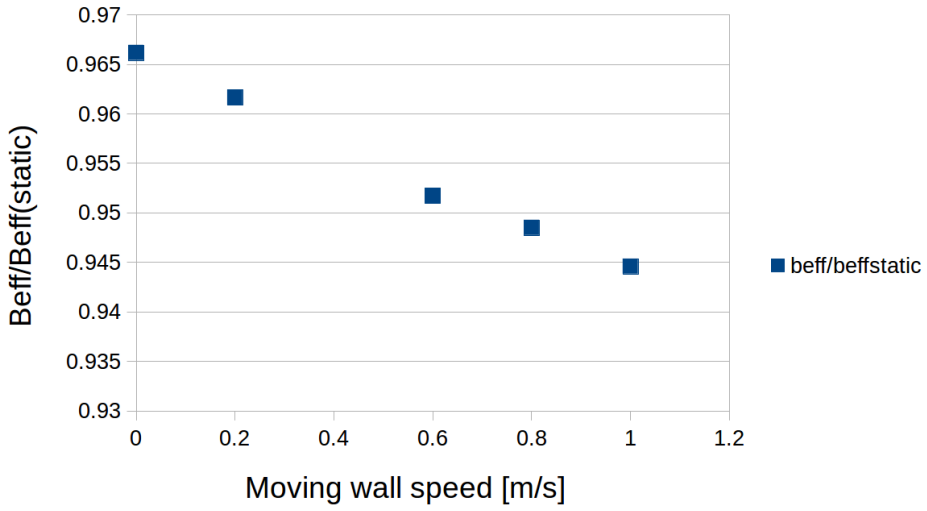

Figure 27: Normalized effective delayed neutron fraction as a function of the moving wall speed, for the fully coupled case.

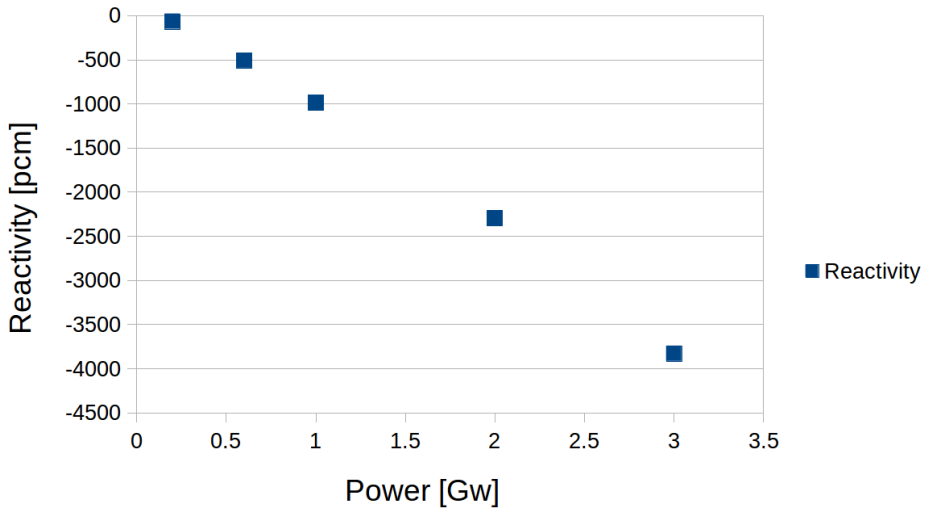

Figure 28: Reactivity as a function of the power level, for the fully coupled case.

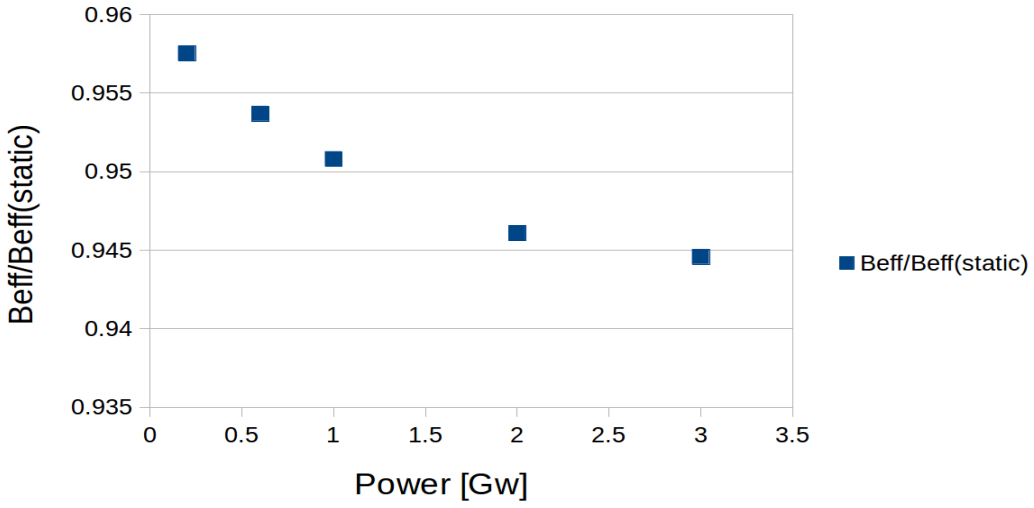

Figure 29: Normalized effective delayed neutron fraction as a function of the power level, for the fully coupled case.

5.3 Evolution along the benchmark's steps and final remarks on the results

The evolution of the velocity field, the temperature distribution and the concentration of two different neutron precursor families are presented in Figure 30, for different stages of the benchmark. Firstly, on the left of the figure, for the static case. Then, for the case in which the velocity field is a fixed input and is not updated during the simulation. After that, moving to the right side of the figure, for the case where buoyant forces are the only responsible for the fuel's motion and, lastly, for the full coupled case. It can be noted how the distributions obtained for the fully coupled scenario are a combination of the fixed velocity field and buoyant driven cases.

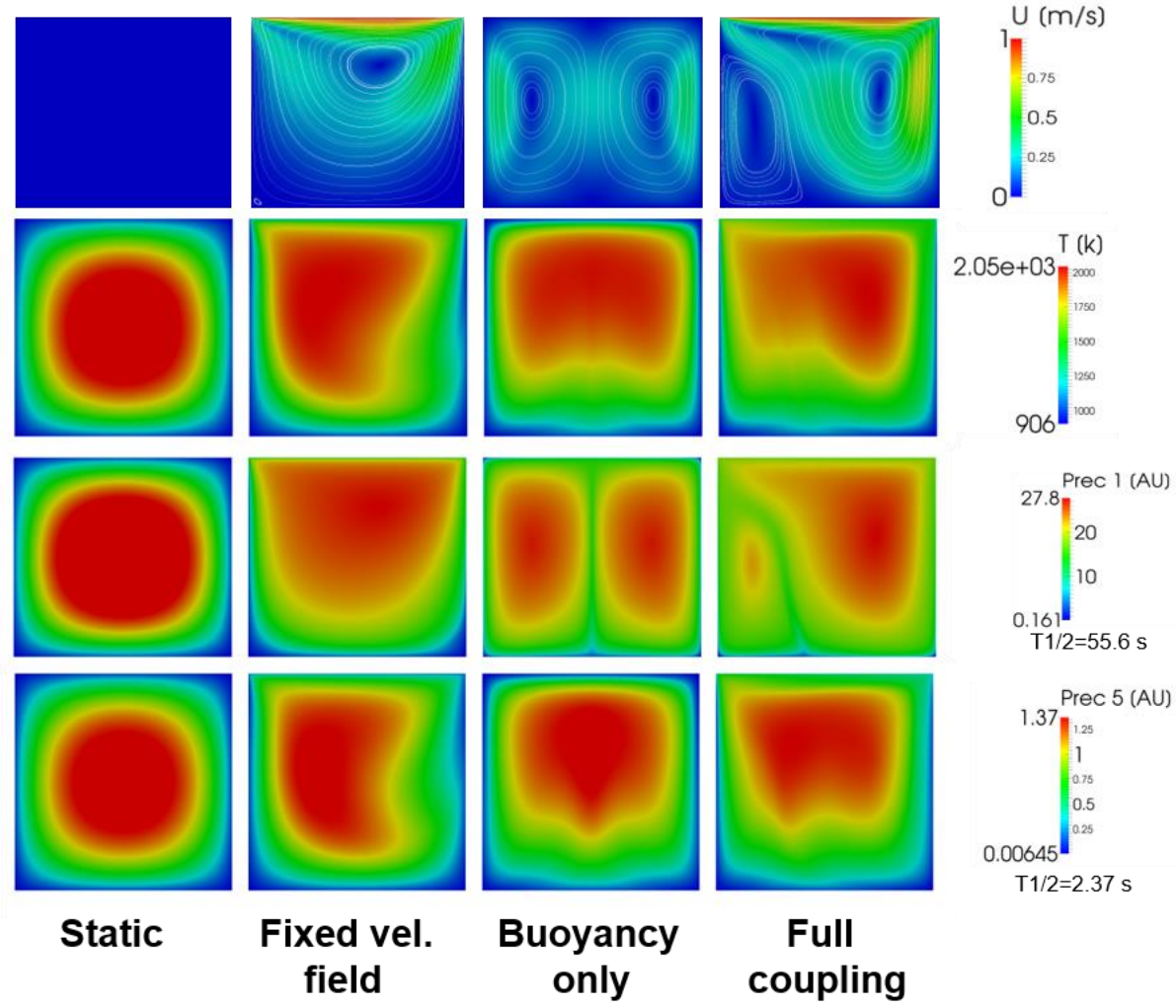

Figure 30: Distributions obtained for different relevant variables for the static case, the case with fixed velocity field, the case with buoyancy driven flow and the fully coupled case.

6 MP coupling for transient analysis

6.1 Current paradigm in MP coupling

One of the most widely used techniques for MP coupling in nuclear reactor calculations is the so called Operator Splitting (OS) technique. It is based on the resolution of the different physics using independent codes that then exchange information at specified points (Miriam Vazquez, 2012). Within this approach, in coupled transient calculations, the time advance of each physical component can be performed simultaneously or in a staggered way. In both cases, only one iteration is performed per time step, thus nonlinearities are not converged. For this reason, the OS methods are said to be "Nonlinearly Inconsistent" (NI).

The equations considered in reactor calculations are stiff because they contain modes with greatly differing time scales. Therefore, only implicit time discretization schemes are employed; the stability requirements of explicit schemes would lead excessively small time steps.

Due to the crude approximations normally used for the time explicit linearization of the nonlinear coupling terms, the current NI-OS coupling schemes are only first-order in time, regardless of the implicit time discretization technique chosen. In consequence, small time steps have to be used to obtain solutions of acceptable accuracy, hence increasing the calculation time.

To overcome the evident limitations of the traditional NI-OS schemes, two different methodologies are envisaged. One is based on small modifications that would make the scheme linearly consistent, and would increase its time accuracy. A different option consists in reformulating the coupled problem into a single block of equations to be solved with, for example, the Newton's method; linearization of NL coupling terms is hence avoided and higher order time discretization schemes could be used. (Jean C. Ragusa, 2009)

In Section 6.2, the first of the two envisaged strategies is explored. The reason for this is that it allows to use the tools already developed in the LPSC, namely the code coupling Serpent and OpenFOAM thermal-hydraulic solvers. This code would need to be modified to perform transient calculations, but some of the capabilities already implemented, such as the precursor transport or the IFP method for calculating the β_{eff}, would be exploited. Another more general reason is that the OS coupling technique allows the use of legacy monophysics calculation codes, that already have years of validation and verification.

6.2 Improving the OS scheme

A straightforward method to improve the OS scheme and increase its time accuracy is the Picard Iterations technique. It basically consists in iterating, within each time step, the non-linear terms until convergence. The iterative procedure implemented in each time step represent a very high computational cost. In addition, if a stochastic code is used for the neutronic component of the calculation, the Piccard Iterations might require unacceptable large CPU times, even if available acceleration algorithms, such as the Aiken's Δ^2 technique, were implemented.

An alternative is found in the predict-evaluate-correct (PEC) methods that employ higher order linearization for the lagged nonlinear terms (Jean C. Ragusa, 2009). To illustrate this, the system of equations representing the coupled problem can be written as

$$y' = L(t)y + N(t, y), \tag{10}$$

where *y* is the solution vector, *L* a linear operator, *N* a nonlinear operator, and *t* represents the time. When an implicit time discretization scheme is employed to solve this system, the resulting equation is nonlinear in the solution vector y^{n+1} at time t^{n+1}. This is,

$$y^{n+1} = f(L(t^n), L(t^{n+1}), N(t^n, y^n), N(t^{n+1}, y^{n+1})), \tag{11}$$

where *f* is a vector valued function. Traditionally, to solve this equation the nonlinear term is explicitly linearized with rough approximations such as

$$N(t^{n+1}, y^{n+1}) = N(t^{n+1}, y^n) + O(\Delta t), \tag{12}$$

that degrade the global convergence order of the algorithm. The PEC method consists in using a prediction $y^{n+1,p}$ for y^{n+1} at time t^{n+1} to use in the linearization of the NL terms with an accuracy in time *q* greater than unity. This is expressed as

$$N(t^{n+1}, y^{n+1}) = N(t^{n+1}, y^{n+1,p}) + O(\Delta t^q), \tag{13}$$

and can be used in a corrector step to calculate y^{n+1} with a system of equations consistent up to order *q*. Normally, the predictor step requires an approximation of the time derivatives of *y*, which are expressed as a linear combination of the solution at previous temporal values.

The PEC method has the advantage of not requiring any iterative process for yielding a more accurate solution than that of the traditional OS scheme.

6.3 Reducing CPU time

Even if non-iterative algorithms, such as the OS with PEC, were employed, the large computational time required for the neutron transport calculation with a Monte Carlo code may limit the viability of this approach for some transients. The CPU time becomes even more limiting for fast transients, like the one following a large and rapid reactivity insertion, due to the small time step that needs to be used to follow the power evolution.

6.3.1 The Quasi-static method

Previous studies on the transient behavior of the MSFR have shown that flux shape is not greatly affected by velocity or temperature distribution (Claudio Nicolino, 2008), which suggests that the Quasi-Static (QS) method could be adopted to reduce the number of neutron transport calculations.

The QS method is based on the factorization of the neutron angular flux into the product of a time dependent amplitude function, *N*, and a shape function φ that also depends on time. This is expressed as

$$\psi(r, \Omega, E, t) = N(t)\, \varphi(r, \Omega, E, t) \tag{14}$$

This factorization is arbitrary, so a normalization condition is employed to make it unique. This condition takes the form

$$\langle \psi_0^*, \frac{1}{v} \psi_0 \rangle = c, \tag{15}$$

where ψ_0 and ψ_0^* represent the steady state flux and adjoint flux (neutron importance), respectively, and $\langle .,. \rangle$ denotes the standard scalar product. The normalization constant *c* can arbitrarily be set to unity.

The idea behind the QS method is to use two different time steps: a small micro step to calculate the fast varying amplitude, and a larger macro step to calculate the shape function. Point Kinetic equations are employed to calculate the evolution of the amplitude with time, and this formalism has already been developed for fluid fueled systems (G. Lapenta, 2001).

There are two different algorithms based on the flux factorization given by Equation 14: The Improved Quasi-static Method (IQM), and the Predictor-Corrector Quasi-static Method (PCQM). In the IQM, the amplitude is computed first, and its followed by a shape calculation. This process is repeated until a convergence criterion based on the normalization condition is fulfilled. On the other hand, in the PCQM, the neutron flux is first calculated in a predictor step and then the amplitude is computed. Finally, the amplitude is used to correct the neutron flux.

In the particular case in which a time consuming Monte Carlo code is used for the transport calculations, the natural choice would be the PCQM, since it is not iterative. The PCQM algorithm has the following steps (Sandra Dulla, 2008):

Step 1. Calculate ψ_0 and ψ_0^* and evaluate the constant *c* of the normalization condition given by Equation 15.

Step 2. Calculate the predicted values of the neutron flux ($\tilde{\psi}_{\Delta t}$) and precursor concentrations ($\tilde{C}_{\mathrm{i},\Delta t}$) after a macro step Δt. To do so, any implicit time integration scheme could be used.
Then, an appropriate shape function can be computed as

$$\varphi(r,\Omega,E,\Delta t) = \frac{\tilde{\psi}(r,\Omega,E,\Delta t)c}{\langle \psi_0^*, \frac{1}{v}\tilde{\psi}\rangle} \tag{16}$$

Step 3. The Point Kinetics parameters are computed using the shape function calculated in Step 2 and the neutron importance function. With these parameters, the amplitude is solved within the micro time scale with step δt, until the its value for $t = \Delta t$ is obtained.

Step 4. With the amplitude calculated in Step 3, the angular flux and the precursor concentrations are corrected.

With Monte Carlo codes, calculating the adjoint neutron flux requires an extra transport calculation. To avoid this, the possibility of using the IFP method implemented in Serpent Monte Carlo code should be studied. This method allows the calculation of direct adjoint-weighted quantities, and its employment would require some modifications in the PCQM algorithm.

To include the non neutronic calculations in the algorithm, a third time step $\delta' t$ is used, such that $\delta t \leq \delta' t \leq \Delta t$. Then, the thermal-hydraulic calculations are performed with a time step $\delta' t$, PK is used every δt to calculate the flux amplitude, and a complete neutron transport calculation is done with a macro time step Δt. In the case of the MSFR, the precursor transport is also evaluated with time step $\delta' t$.

To improve the accuracy of the flux calculation, the same options than in Section 6.2 are available: an iterative algorithm or a PEC method with higher order linearization of the NL lagged terms. In order to reduce the computational time as much as possible, the non-iterative PEC method should be employed. However, a careful analysis of this implementation should be conducted since, depending on how large the macro step Δt employed is, an iterative procedure might be necessary to obtain acceptable results; i.e., if two successive transport calculation are too distant in time, the PEC method might not be enough to yield the desired accuracy.

6.3.2 Time step selection

It is evident that the selection of the step size must be carefully done to achieve an acceptable balance between CPU time and accuracy. Using a constant step size is far from optimal, since it is clear that in any given transient simulation some stages require a finer time discretization than others, depending on the magnitude of the time derivatives involved. To address this, two strategies are commonly employed: adaptive and predictive time step (Cyril Patricot, 2016).

In the adaptive time-step technique, any given interval that does not meet the accuracy requirement is recalculated using a smaller time step. For this particular problem, the accuracy requirement could be based on the normalization condition defined by Equation 15. On the other hand, a larger time step is used for an interval following one in which the error was found to be below a certain threshold.

In the predictive time-step technique, a time step size is conveniently chosen for each interval, thus no recalculations are needed. In this case, an adequate criterion for choosing the macro step size could be the reactivity change between two consecutive micro steps δt; if this difference is found to be larger than a selected threshold, a flux calculation is performed.

Once again, the aim of reducing the number transport calculations makes the predictive time-step technique, that does not require any recalculation of a time interval, the natural option. This is supported by previous studies where this technique produced better results than the adaptive time-step for fast transient coupled MP calculations (Cyril Patricot, 2016).

7 Conclusions and future work

In the present work, a benchmark designed in the LPSC to evaluate the performance of different codes in coupled MP scenarios was studied. In Section 3, a detailed description of the benchmark is given, and the different phenomena to be investigated in each of its stages are specified. To do this, the general characteristics of the MSFR, which constitutes a framework for the development of the benchmark, had been previously described and the most relevant MP coupling phenomena identified in Section 2.

In addition, a coupled MP calculation code base in Serpent and OpenFOAM, also developed in the LPSC, was studied. It was compiled to run in parallel in a Linux platform, and it was described in Section 4. This code was also modified to increase its flexibility to perform the calculations of the benchmark, and its general structure and Input/Output system was documented.

The benchmark was performed with the above mention code, and the results presented in Section 5 of this work show that it meets its objective of providing a simulation scenario where all the fundamental MP coupling phenomena of the MSFR are present and where the different elements can be progressively introduced and studied independently. Even subtle effects related to, for example, the behavior of the effective delayed neutron fraction could be reproduced and isolated to be studied, despite the simplicity of the problem modeled. This simplicity made possible that all the calculations needed were performed within reasonable time, regardless of the relatively slow Monte Carlo code used for neutronic calculations.

Furthermore, the obtained results, which are physically consistent, demonstrated that the code developed in the LPSC is capable of capturing the coupling mechanisms presented in the benchmark and, thus, that it most likely is a useful design tool for the MSFR if some modifications are made, though more rigorous numerical benchmarks would be needed to firmly determine this.

Finally, in Section 6 the current paradigm in coupled MP transient calculations for nuclear reactors was briefly reviewed, and possibilities of increasing its accuracy and reducing the computational times involved were explored. Particularly, a strategy was outlined for transient calculations maintaining the same approach that had been used for steady state calculations, which is based on coupling Serpent with thermal-hydraulics solvers implemented in OpenFOAM. The proposed strategy is based on the Predictor-Corrector Quasi-static Method with predictive time step selection and a Predict-Evaluate-Correct algorithm with higher order linearization for the lagged nonlinear coupling terms.

Several different lines of work are left open to continue what has been done so far. Firstly, further investigation of some of the hypothesis adopted in Section 2.2 is needed, in particular in the coupling between turbulence and neutronics and in the 3D coupling effects, that were considered to have lesser importance. On the other hand, the error encountered in the code in the calculation of the neutron precursors concentrations, mentioned in Section 5.2.1, should be corrected, and its impact in the results examined. Lastly, the applicability of the strategy outlined in Section 6 should be evaluated. In particular, the possibility of modifying the PCQM algorithm to profit from the IFP method for adjoint weighting quantities implemented in Serpent should be explored, as well as the accuracy achieved by using a non-iterative PEC method to deal with the non-linear terms in the MP coupling when the macro time step is increased.

8 References

CFD Direct Ltd. (2015). *OpenFOAM: The Open Source CFD Toolbox. Programmer's Guide.*

CFD Direct Ltd. (2015). *OpenFOAM: The Open Source CFD Toolbox. User Guide.*

Claudio Nicolino, G. L. (2008). Coupled dynamics in the physics of molten salt reactors. *Annals of Nuclear Energy*.

Cyril Patricot, A.-M. B. (2016). INTERESTS OF THE IMPROVED QUASI-STATIC METHOD FOR MULTI-PHYSICS CALCULATIONS ILLUSTRATED ON A NEUTRONICS-THERMOMECHANICS COUPLING . *PHYSOR 2016- Unifying Theory and Experiments in the 21st Century.*

G. Lapenta, F. M. (2001). Point kinetic model for fluid fuel systems. *Annals of Nuclear Energy*.

JaakkoLeppanen, V. V. (2014). UNSTRUCTURED MESH BASED MULTI-PHYSICS INTERFACE FOR CFD CODE COUPLING IN THE SERPENT 2 MONTE CARLO CODE. *PHYSOR 2014- The Role of Reactor Physics toward a Sustainable Future.*

Jean C. Ragusa, V. S. (2009). Consistent and accurate schemes for coupled neutronics thermal-hydraulics reactor analysis. *Nuclear Engineering and Design*.

Laureau, A. (2015). *Développement de modèles neutroniques pour le couplage thermohydraulique du MSFR et le calcul de paramètres cinétiques effectifs.*

Leppänen, J. (2007). *Development of a New Monte Carlo Reactor Physics Code.*

Leppänen, J. (2009). ON THE USE OF THE CONTINUOUS-ENERGY MONTE CARLO METHOD FOR LATTICE PHYSICS APPLICATIONS. *International Nuclear Atlantic Conference.*

Leppänen, J. (2015). *Serpent – a Continuous-energy Monte Carlo Reactor Physics Burnup Calculation Code.*

LPSC. (2014). *Benchmark: preliminary description of the steps for the steady-state phase.* Grenoble.

Manuele Aufiero, A. C. (2012). Multi-physics modelling of the Molten Salt Fast Reactor using OpenFOAM. *7th OpenFOAM Workshop.* Darmstadt.

Manuele Aufiero, M. B. (2014). Calculating the effective delyaed neutron fraction in the Molten Salt Fast Reactor: Analytical, deterministic and Monte Carlo approaches. *Annals of Nuclear Energy*.

Merle-Lucotte, E. (2015). Presentation Presentation of the MSFR reactor reactor concept. *Workshop SERPENT and Multiphysics.*

Miriam Vazquez, H. T.-T.-F. (2012). Coupled neutronics thermal-hydraulics analysis using Monte Carlo and sub-channel codes. *Nuclear Engineering and Design*.

OECD Nuclear Energy Agency. (2014). *Technology Roadmap Update for Generation IV Nuclear Energy Systems.*

Samofar. (February de 2016). Obtained from http://samofar.eu/concept/

Sandra Dulla, E. H. (2008). The Quasi-static method revisited. *Progress in Nuclear Energy*.